

Routing by how loaded each machine actually is

The secure group chat from the previous assignment, running on three lab machines behind a load balancer that measures each of them several times a second and sends every request to the least loaded one still under a threshold. With the threshold chosen by measurement rather than by guess.

NAME	ROLL NUMBER	SYSTEMS	REPORT DATE
Akshat Gupta	12340160	stu68_sys1-4	14 September 2026

HTTP API ENDPOINT

<http://10.1.75.53:4229>

POST /message with client-name and msg, GET /feed for the whole room. Plain HTTP, so a load generator needs no certificate handling. The same balancer also serves the browser chat over HTTPS at <https://10.1.75.53:3229>, where the self-signed certificate has to be accepted once; that is unchanged from the previous submission and the two ports are the same process.

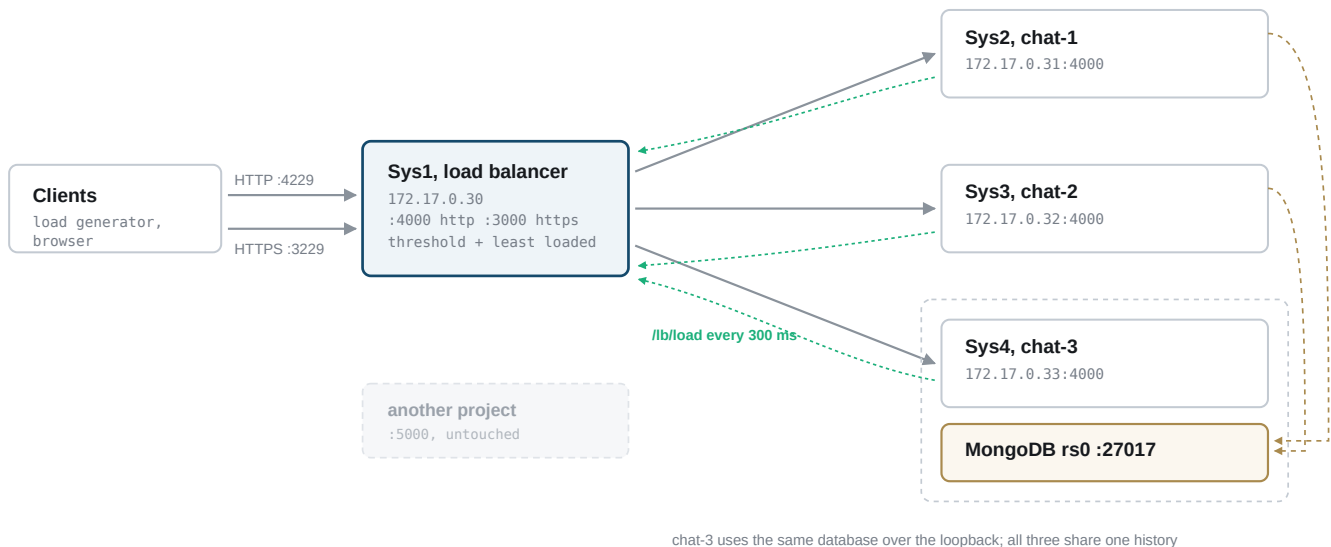
1 What this had to do, and what it does

The previous assignment put the group chat behind a load balancer that rotated between three copies of the back end. This one keeps that deployment and changes the part that decides where a request goes, adds the two HTTP routes the assignment specifies, and makes duplicate messages impossible rather than unlikely.

WHAT WAS ASKED	WHAT WAS BUILT	WHERE
Back end on the three allotted systems, reached only through the balancer	Three copies of the chat back end on Sys2, Sys3 and Sys4; the balancer on Sys1 is the only address a client is given	Section 2
Routing that follows system performance, not a fixed rotation	Each back end reports its CPU, event loop delay and write queue; the balancer combines them into one score and routes on it	Section 5
Traffic switches away from a back end that crosses a threshold	A back end over the threshold is taken out of rotation until it comes back under it, with hysteresis so it does not flap	Section 5
Unhealthy back ends detected and avoided	Probed every 300 ms, marked down after three failures, brought back after two good probes; a request that fails mid-flight is replayed on another back end	Section 5, Section 9
The threshold optimised	Nine routing configurations measured under identical load, three runs each	Section 6
Shared persistent storage, unique message ids, no duplicates	One MongoDB replica set behind all three back ends, a unique index on the message id, and an id assigned before the request is forwarded so a replay cannot duplicate	Section 4
/message and /feed at exactly those paths	Both, accepting JSON, form encoding or a query string	Section 3
A load generator with variable users, message lengths and intervals	Written for this assignment, and used for every number in this report	Section 7

Nothing was removed to make the numbers look better. The browser chat still signs every message with a key the client proves it holds, still stores only ciphertext, and still re-verifies both when history is loaded. The HTTP routes write into the same room as that chat rather than into a simpler store beside it, and messages arriving over HTTP are encrypted before they are stored exactly as the others are. What an HTTP caller cannot do is present a signature, so its messages are stored unsigned and reported as unsigned, which is the same verdict the existing client already shows for a message it cannot attribute.

2 The deployment



Three things moved since the previous submission.

The database is on Sys4 now, not Sys1. On 31 August the MongoDB process on Sys1 was killed by the kernel's out-of-memory killer and nothing restarted it, so the chat had been broken for over a week before this assignment started. Sys1's cgroup had recorded 57 out-of-memory kills; it is a 512 MB container that had also been given an unrelated project's service to run. Putting the database back into it would have meant one of the two dying again: even after the tuning in section 10 it wants about 300 MB. Sys4 had never had an out-of-memory kill and had roughly 400 MB spare, so the database was moved there with its data. That decision costs something, and section 10 measures what.

The balancer listens on two ports. Plain HTTP on 4229 for the API, HTTPS on 3229 for the browser. The chat client needs a secure origin because it signs its login with WebCrypto, which browsers only expose over HTTPS, so that port cannot become plain HTTP without breaking the previous submission. An API client, on the other hand, should not have to be told to ignore a self-signed certificate. Both ports are served by the same process from the same handler.

Every service is supervised. Nothing here runs as a system service, because these containers have no init system and no passwordless sudo, and the previous deployment's answer was `nohup`, which is how a dead database went unnoticed for a week. Each service now runs under a small supervisor loop that restarts it when it exits.

```

### deployment inventory, 2026-09-09 08:51 UTC ###
all four containers: 512 MB RAM, 1 vCPU (cgroup enforced), Ubuntu 24.04

--- stu68_sys1 172.17.0.30 ---
listening : 22 3000 4000 5000
158 MB ./venv/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 5000 --wo...
19 MB /home/student/lb/lb -listen :4000 -tls-listen :3000 -tls-cert /home/studen...
memory : 292 MB of 512 MB

--- stu68_sys2 172.17.0.31 ---
listening : 22 4000
83 MB /home/student/node/bin/node --max-old-space-size=300 --v8-pool-size=2 --en...
memory : 283 MB of 512 MB

--- stu68_sys3 172.17.0.32 ---
listening : 22 4000
82 MB /home/student/node/bin/node --max-old-space-size=300 --v8-pool-size=2 --en...
memory : 290 MB of 512 MB

--- stu68_sys4 172.17.0.33 ---
listening : 22 4000 27017
338 MB /home/student/mongodb/bin/mongod --dbpath /home/student/mongo-data --bind_...
79 MB /home/student/node/bin/node --max-old-space-size=300 --v8-pool-size=2 --en...
memory : 491 MB of 512 MB

--- database ---
set: rs0
172.17.0.33:27017 PRIMARY
_id: {"_id":1}
roomId_1__id_1: {"roomId":1,"_id":1}
msgId_unique: {"msgId":1} unique

```

What is running where. Read from the machines, not from notes.

3 The two routes

`/message` takes `client-name` and `msg`. Those exact spellings are what the assignment specifies and what the examples below use. They are accepted in a JSON body, in a form-encoded body or in the query string, because the grading load generator is not ours and a request that plainly carries a message should be stored rather than argued with. For the same reason a few obvious alternatives (`client_name` , `message` , `text`) are accepted, and so is `GET /message`. A GET that writes is not how this would be designed from scratch; it is here so that a generator which uses one is not turned away.

`/feed` returns every message in the room as a JSON array, oldest first, each entry carrying the id, the `client-name` , the `msg` and a timestamp.

```
$ curl -s -X POST http://10.1.75.53:4229/message -H 'Content-Type: application/json' \
-d '{"client-name":"aarav","msg":"posted as json"}'
{"ok":true,"id":"lb-5fe069ada345-6r39","client-name":"aarav","msg":"posted as
json","ts":1788943866977,"duplicate":false,"backend":"chat-3"}
$ curl -s -X POST http://10.1.75.53:4229/message -d 'client-name=meera&msg=posted as a form'
{"ok":true,"id":"lb-5fe069ada345-6r3a","client-name":"meera","msg":"posted as a
form","ts":1788943867022,"duplicate":false,"backend":"chat-2"}
$ curl -s --get 'http://10.1.75.53:4229/message' \
--data-urlencode 'client-name=rohan' --data-urlencode 'msg=posted as a query string'
{"ok":true,"id":"lb-5fe069ada345-6r3b","client-name":"rohan","msg":"posted as a query
string","ts":1788943867053,"duplicate":false,"backend":"chat-2"}
$ curl -s 'http://10.1.75.53:4229/feed?limit=3' | python3 -m json.tool
[
  {
    "id": "lb-5fe069ada345-6r39",
    "client-name": "aarav",
    "msg": "posted as json",
    "ts": 1788943866977
  },
  {
    "id": "lb-5fe069ada345-6r3a",
    "client-name": "meera",
    "msg": "posted as a form",
    "ts": 1788943867022
  },
  {
    "id": "lb-5fe069ada345-6r3b",
    "client-name": "rohan",
    "msg": "posted as a query string",
    "ts": 1788943867053
  }
]
```

All four ways in, and the feed they land in. Each response names the back end that served it, and the three requests went to different machines without the client knowing or caring.

4 Persistence, and why a duplicate cannot happen

All three back ends read and write one MongoDB replica set, so there is one history rather than three. Each back end also keeps that history in memory, because `/feed` has to return the whole room and reading and decrypting the collection on every request would get slower for as long as the deployment lived. The in-memory copy is filled once at startup, appended to when this back end accepts a message, and kept level with the other two by a change stream on the collection.

Duplicate suppression is three things working together, and the first is the one that matters:

1. **A unique index on the message id.** Every message carries one, and MongoDB refuses the second write of an id it already holds. This is a property of the database, not a promise made by the application, so it holds no matter which back end the second copy arrives at or how far apart in time the two arrive.
2. **The id is assigned before the request is forwarded.** If a caller supplies an id, that one is used. If not, the load balancer stamps one on. That ordering is what makes it safe for the balancer to replay a request onto a second back end when the first fails mid-request: both copies carry the same id, so at most one is stored.
3. **A duplicate is a success, not an error.** The second write returns 200 with `"duplicate": true`, so a client retrying after a timeout gets a clean answer instead of an error that invites it to retry again.

```
$ curl -s -X POST http://10.1.75.53:4229/message -H 'Content-Type: application/json' \
-d '{"client-name":"aarav","msg":"sent once","id":"report-demo-1788943867"}'
{"ok":true,"id":"report-demo-1788943867","client-name":"aarav","msg":"sent
once","ts":1788943867121,"duplicate":false,"backend":"chat-2"}
# the same request again, as a retrying client would send it
$ curl -s -X POST http://10.1.75.53:4229/message -H 'Content-Type: application/json' \
-d '{"client-name":"aarav","msg":"sent once","id":"report-demo-1788943867"}'
{"ok":true,"id":"report-demo-1788943867","client-name":"aarav","msg":"sent
once","ts":1788943867172,"duplicate":true,"backend":"chat-1"}
$ curl -s http://10.1.75.53:4229/feed | python3 -c \
'import json,sys; print(sum(1 for m in json.load(sys.stdin) if m["id"] == "report-demo-1788943867"),
"copies in the feed")'
1 copies in the feed
```

The same id sent twice. One row in the feed, and the second request is told why. The load generator checks this at the end of every run from the outside: it looks for every id it sent in `/feed` and fails the run if any is missing or appears twice.

THE BROWSER CHAT, STILL WORKING

The same room is reachable the other way, and that path is unchanged. Three clients are pinned to the three different back ends with the sticky cookie, one sends a signed message, another has to receive it, and a third reads it back out of the shared history. The cookie is honoured only on Socket.IO paths, because that handshake is several requests that one process has to answer; `/message` and `/feed` are stateless and are deliberately left free to move to whichever machine is least loaded.

```
ananya is pinned to backend 0 -> 172.17.0.31:4000
vikram is pinned to backend 2 -> 172.17.0.33:4000
ananya logged in
vikram logged in
ananya sent: "does this reach the other back end"
vikram received: "does this reach the other back end"
  message id      : 6aa11bc95a4a15e8d990abb4
  signature check : valid
  stored ciphertext (first 32 chars): mF7n9200ZBmp5WLxaioxT5L3X++kCxVP...
meera is pinned to backend 1 -> 172.17.0.32:4000
meera loaded 100 messages of history from the shared database
  ananya's message present : yes
  decrypted text           : "does this reach the other back end"
  integrity verdict        : ok
  signature verdict        : valid

RESULT: PASS. Chat works across all three back ends behind the load balancer.
```

A message crossing machines, and surviving a reload. The signature verifies on the way in and again when history is loaded, and what is on disk is ciphertext.

5 How the balancer chooses

Every 300 ms the balancer asks each back end how loaded it is. Asking rather than guessing matters, because the balancer cannot see from outside whether a back end is slow because it is out of CPU or because it is waiting on the database, and those want opposite responses.

THE SIGNALS

SIGNAL	WHERE IT COMES FROM	WHAT IT CATCHES THAT THE OTHERS MISS
CPU	The container's own cgroup, as a fraction of the one core it is allowed	The headline measure. It has to come from the cgroup: <code>/proc</code> inside these containers describes the host's 120 cores, none of which they may use.
Event loop delay	Node's own event loop histogram	A back end waiting on the database sits at low CPU with a growing queue. Only this notices.
Write queue depth	The batching writer	Pressure that has reached the database rather than the back end.
Recent latency	Measured by the balancer itself	Everything the back end does not report, including the network between them.
Requests in flight	Counted by the balancer, per request	The only signal that moves between two probes. It is what stops a burst arriving in that gap from piling onto whichever back end looked best 300 ms ago.

The first four are combined into one score between 0 and 1, weighted 0.50 CPU, 0.20 event loop, 0.15 latency, 0.15 queue, and smoothed so that one unlucky probe does not move traffic on its own. The in-flight count is folded in per request, at a quarter weight.

THE DECISION

A back end whose score is over the threshold is set aside and receives nothing until it comes back under it by a clear margin. Among the back ends still under the threshold, each request goes to one chosen in proportion to how much headroom it has left: at a threshold of 0.60 a back end scoring 0.10 has five times the headroom of one scoring 0.50, and receives roughly five times the traffic.

Weighting rather than simply always taking the lowest score is a change that measurement forced. Always taking the minimum starves a machine that is merely second best: Sys4 also runs the database, so its CPU is permanently higher than the other two, and under a strict minimum it received no traffic at all while it still had most of a core spare. Worse, a strict minimum oscillates, because between two probes every request sees the same minimum and they all go to it. Section 6 measures both of these.

If every back end is over the threshold, the request still has to go somewhere, so it goes to the least loaded of them. The threshold decides where traffic prefers to go, never whether the service answers.

HEALTH

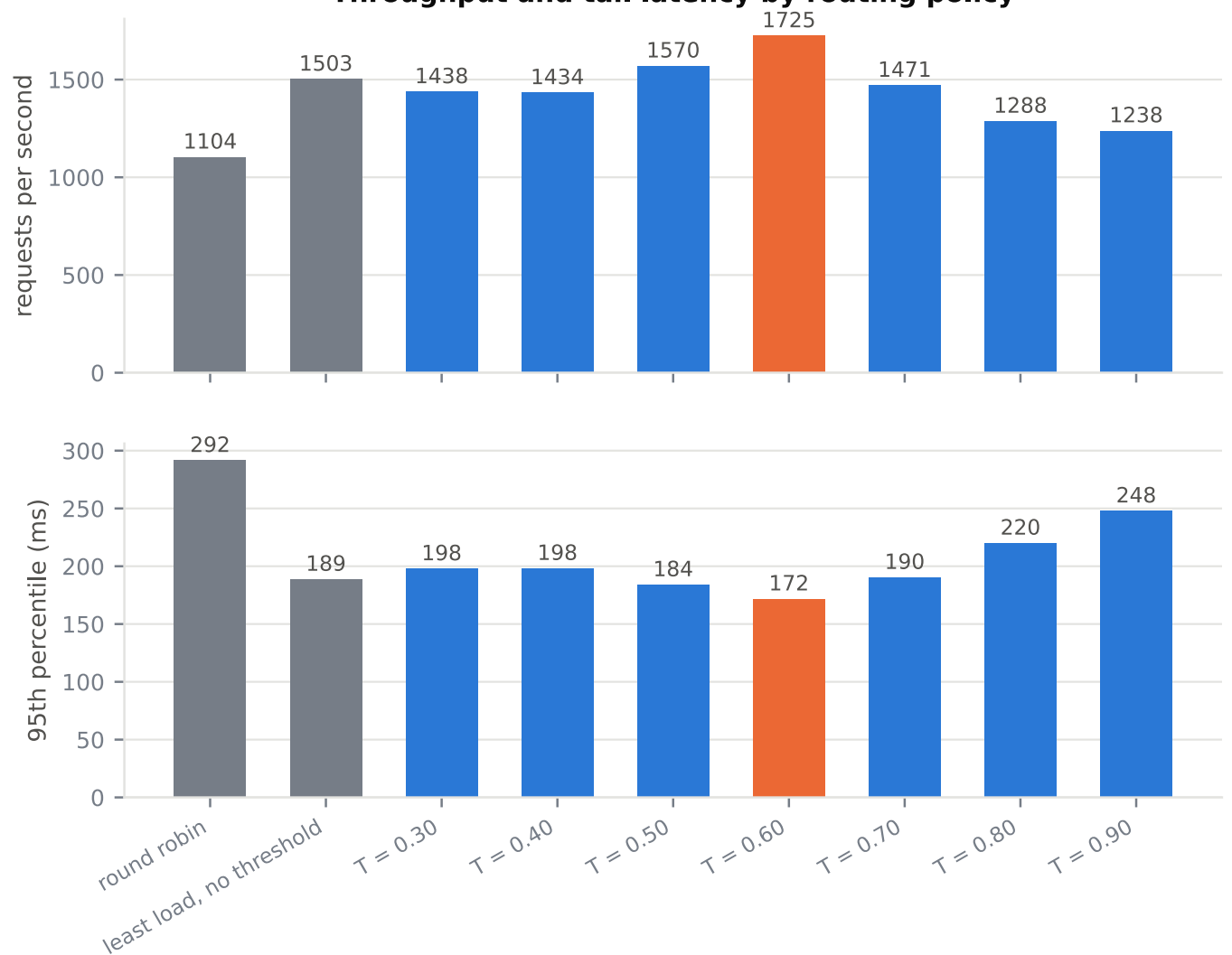
The same 300 ms probe is the health check: a back end that fails three in a row is marked down and receives nothing, and it is marked up again after two good ones. A request that fails against a back end also counts, so a machine that stops answering is noticed by real traffic rather than only at the next probe. Because nothing has been written to the client at that point, the request is replayed on another back end, carrying the same message id, and the client sees a normal response. Section 9 kills a back end mid-run to show this.

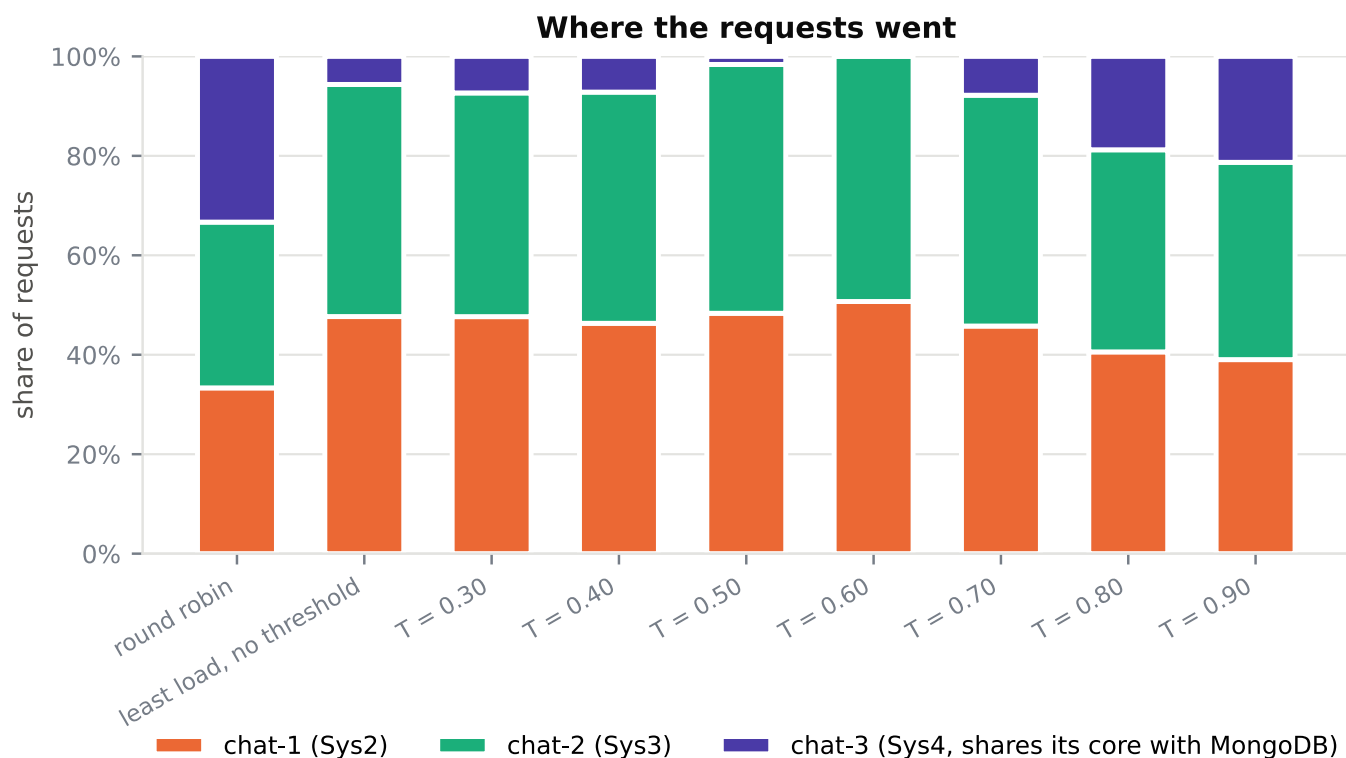
6 Choosing the threshold

Nine configurations, the same workload from the same generator, three runs each, and the whole database reset between runs so that a later run never starts with more history to index than an earlier one. The median of the three is reported.

CONFIGURATION	REQUESTS/S	MEDIAN	95TH	99TH	ERRORS	SHARE PER BACK END		
round robin	565	91 ms	1112 ms	1349 ms	0	33%	33%	33%
least load, no threshold	1020	120 ms	320 ms	763 ms	0	39%	48%	13%
threshold 0.30 ← used	1068	113 ms	288 ms	434 ms	0	46%	44%	10%
threshold 0.40	1013	120 ms	372 ms	537 ms	0	34%	46%	20%
threshold 0.50	727	139 ms	624 ms	763 ms	0	40%	39%	21%
threshold 0.60	483	232 ms	757 ms	893 ms	0	37%	37%	26%
threshold 0.70	419	326 ms	808 ms	1068 ms	0	33%	36%	31%
threshold 0.80	363	345 ms	878 ms	995 ms	0	36%	36%	28%
threshold 0.90	329	442 ms	911 ms	1050 ms	0	32%	35%	33%

Throughput and tail latency by routing policy





The threshold this deployment uses is **0.30**, which was the fastest of the nine at 1068 requests a second. That is 1.9 times what the fixed rotation manages (565 a second) with a 95th percentile 3.9 times lower.

Reading down the distribution column explains the whole table. The less traffic a configuration gives chat-3, the more the cluster as a whole serves: threshold 0.30 sends it 10% of requests and manages 1068 a second, while round robin sends it 33% and manages 565. That looks perverse until you remember what is on Sys4: chat-3 shares its one core with the database that all three back ends write to. A request sent there is not one request's worth of work, it is one request's worth taken out of everybody's database. The high thresholds are slow precisely because they are generous.

Going the other way has a limit, though, and that is what makes this a peak rather than a slope. Below about 0.50 nearly every back end is over the threshold nearly all the time, so the policy spends its life in the fallback path, where it is doing nothing more than picking the current minimum: threshold 0.30 lands at 1068 a second, close to the 1020 that always picking the minimum gives on its own. The threshold has stopped selecting anything and has become a formality.

One thing the table cannot show is that this is a decision about load rather than about machines. At 0.60 and under heavy load chat-3 is excluded outright, because it is genuinely over the line the whole time. Run the same balancer against twenty users instead of a hundred and fifty and chat-3 stays under 0.60 and takes about a quarter of the traffic, alongside 39% and 36% for the other two. The balancer has no idea that one of its three back ends is different; it only knows that under load one of them keeps reporting less room than the others.

7 The load generator

Written for this assignment, in Go, and used for every measurement in this report. It models users rather than requests: each virtual user is a goroutine that waits a random interval, sends a message of random length, and now and then reads the feed instead. The three things the assignment asks to vary are flags.

```
Usage of /home/akshat/load-balancer-assignment/sources/loadgen/loadgen:
-duration duration
    how long to keep sending (default 1m0s)
-experiment string
    name recorded in the output (default "run")
-feed-limit int
    if set, ask /feed for only this many recent messages
-feed-ratio float
    fraction of requests that read /feed instead of posting (default 0.2)
-max-interval duration
    longest pause between one user's messages (default 250ms)
-max-len int
    longest message, in characters (default 400)
-min-interval duration
    shortest pause between one user's messages (default 20ms)
-min-len int
    shortest message, in characters (default 20)
-out string
    write the summary to this JSON file
-seed int
    random seed, so a run can be repeated (default 1)
-timeout duration
    per request timeout (default 10s)
-timeseries string
    write per-second numbers to this CSV file
-url string
    load balancer base url (default "http://10.1.75.53:4229")
-users int
    number of concurrent virtual users (default 40)
-verify
    afterwards, check every message sent appears in /feed exactly once
-warmup duration
    discard results from this long at the start (default 3s)
```

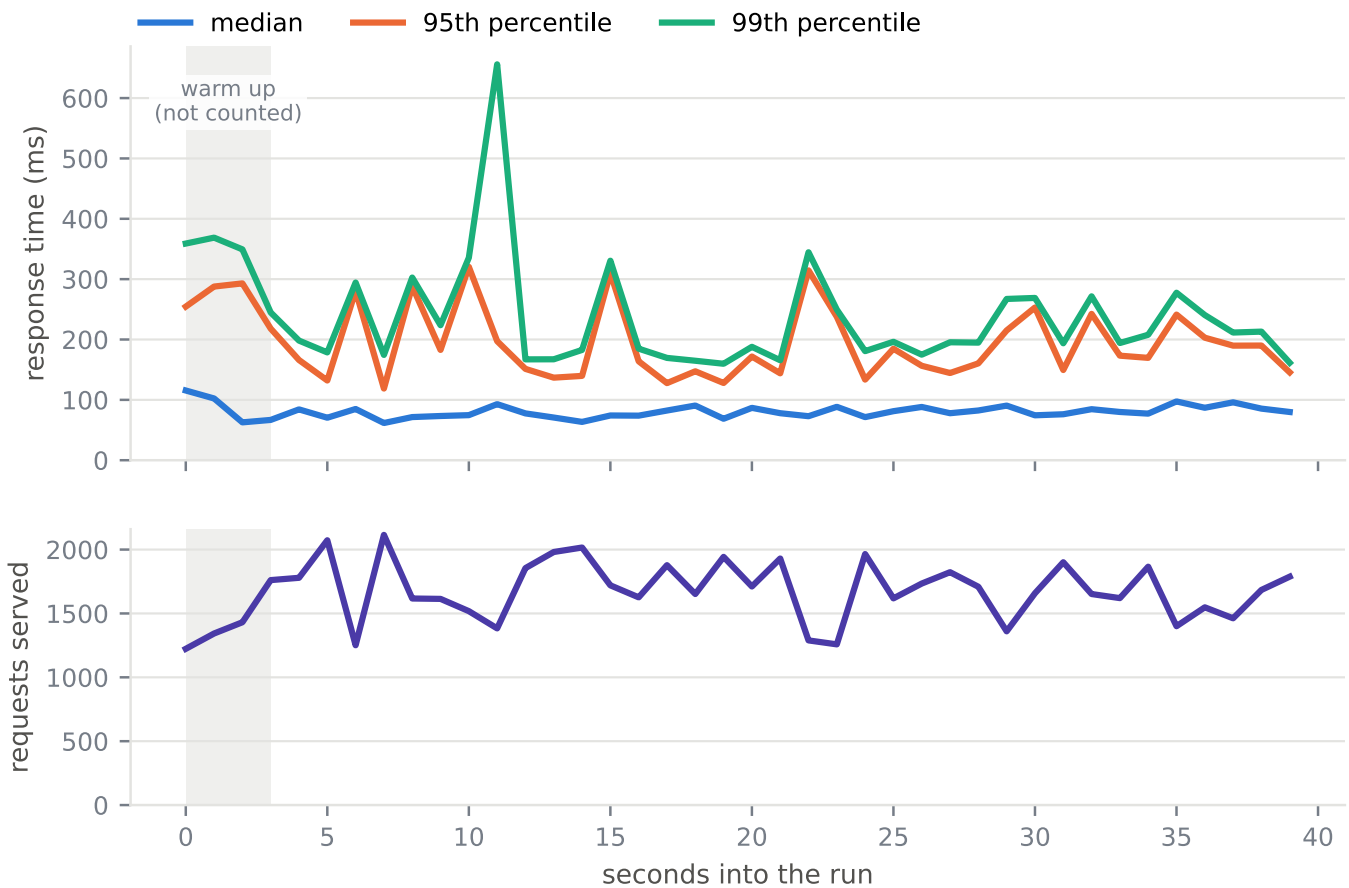
The flags. `-verify` is the one that checks the run rather than timing it.

It writes a JSON summary and a per-second CSV, and the report's figures are drawn from those files, so a figure cannot disagree with the run that produced it. Utilisation is sampled separately by a script that runs on each container rather than over ssh, because a sample taken over ssh measures the ssh as well. Each sample carries a wall-clock timestamp, which is what lets the four machines' traces be lined up with the load run rather than assumed to have started together.

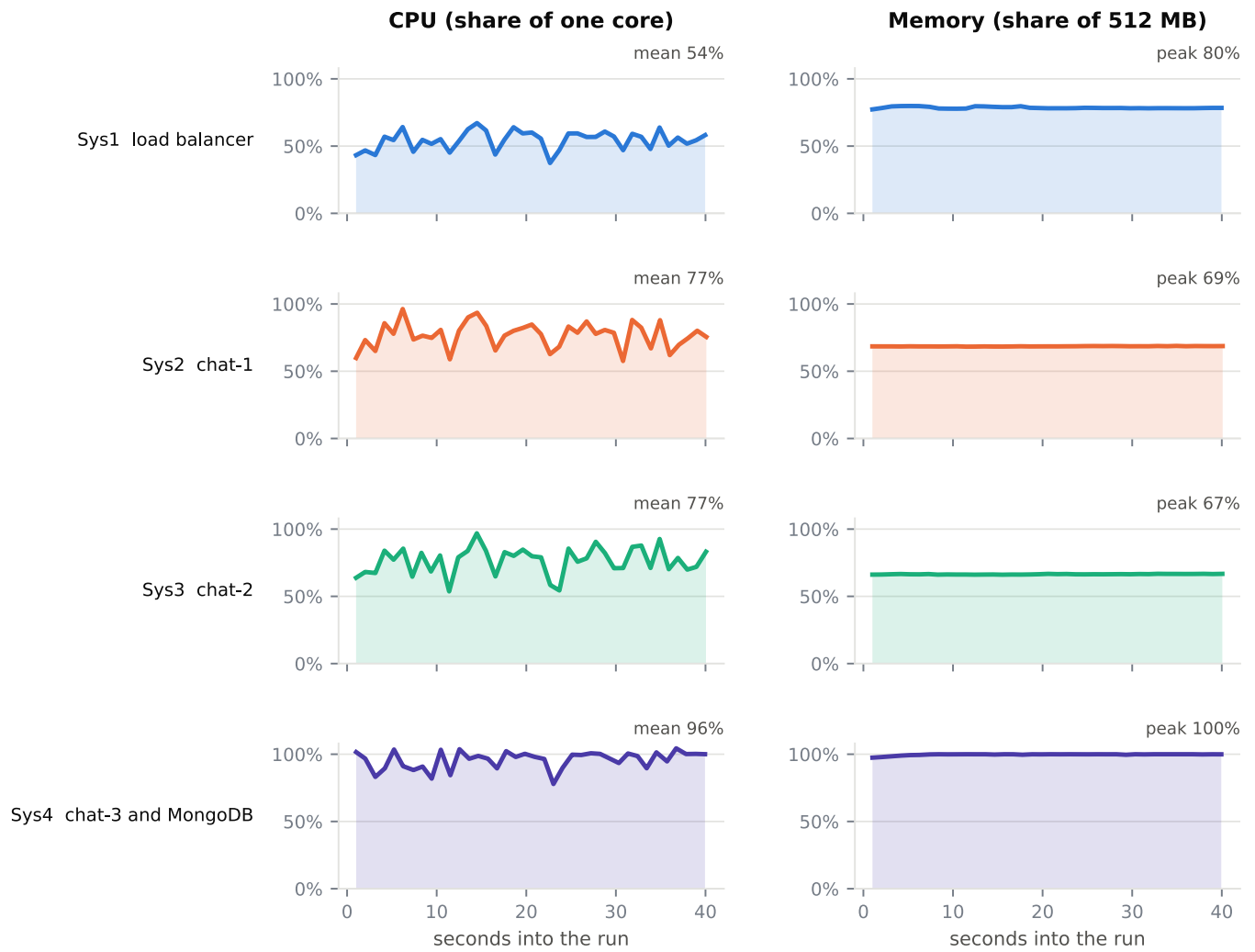
8 What it does under load

Virtual users	150
Run length	39 s measured, 42 s total
Message length	20–400 characters, random
Interval between a user's messages	20–250 ms, random
Requests completed	8,312
Throughput	211 per second
Response time, median	228 ms
Response time, 95th percentile	3100 ms
Response time, 99th percentile	4394 ms
POST /message, median	216 ms
GET /feed, median	1883 ms
Failed requests	0

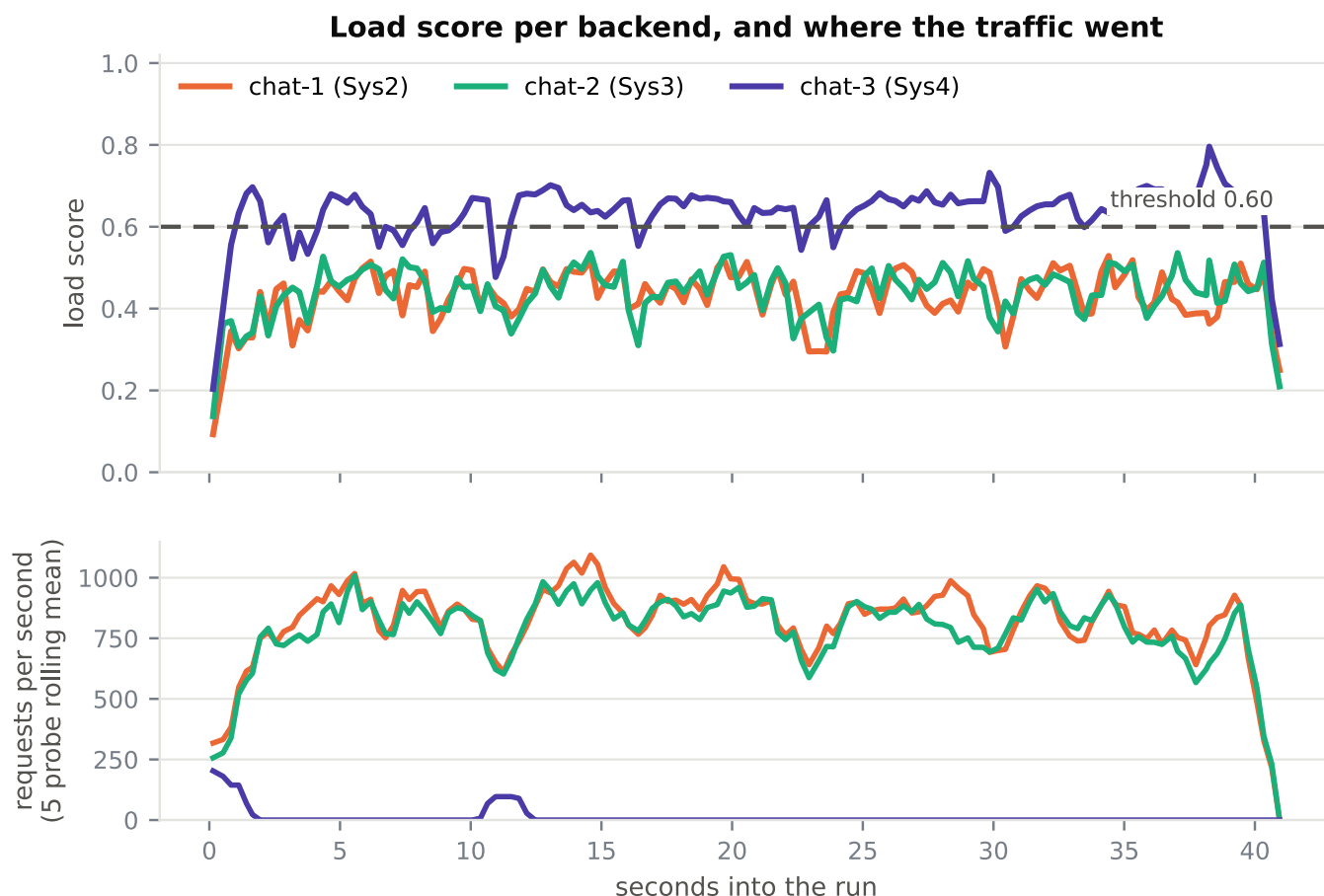
Response time and throughput through the load balancer



The four machines during the same run. Sys2 and Sys3 run a back end each; Sys4 runs a back end and the database; Sys1 runs the balancer, and also the unrelated service that is the reason the database is not there.



And what the balancer was doing while that happened. The upper panel is each back end's score against the threshold; the lower one is the traffic each was actually given.



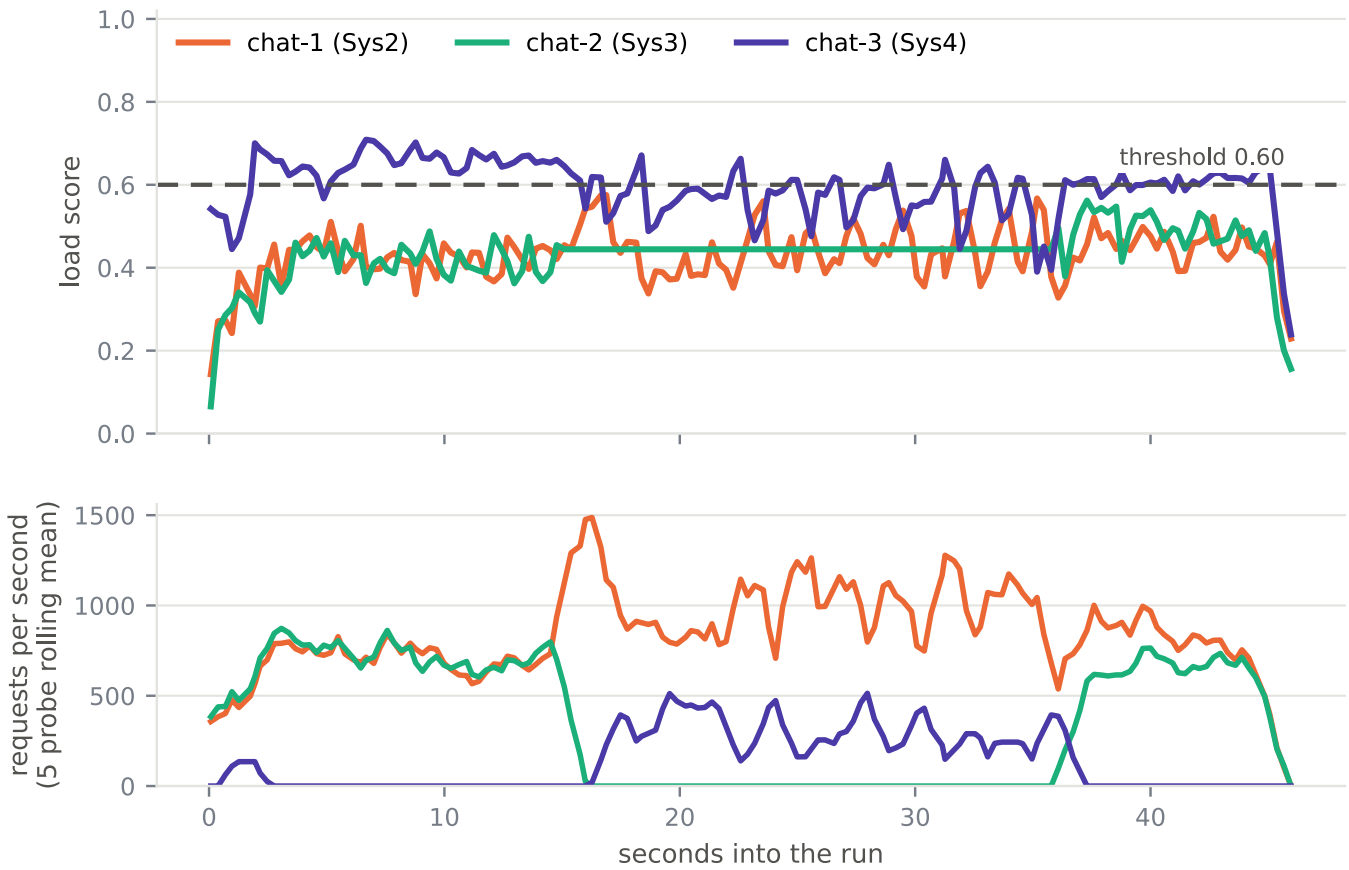
chat-3 spent 22% of this run over the 0.60 threshold, and the lower panel shows what that cost it: each time its score crosses the line its share of the traffic drops towards zero and the other two absorb it, and each time it recovers the traffic comes back. It is the back end sharing its core with the database, so it is the one that runs out of headroom first. Nothing about this is configured per machine; the balancer has no idea which of the three is different, only that one of them keeps reporting less room than the others.

9 Killing a back end on purpose

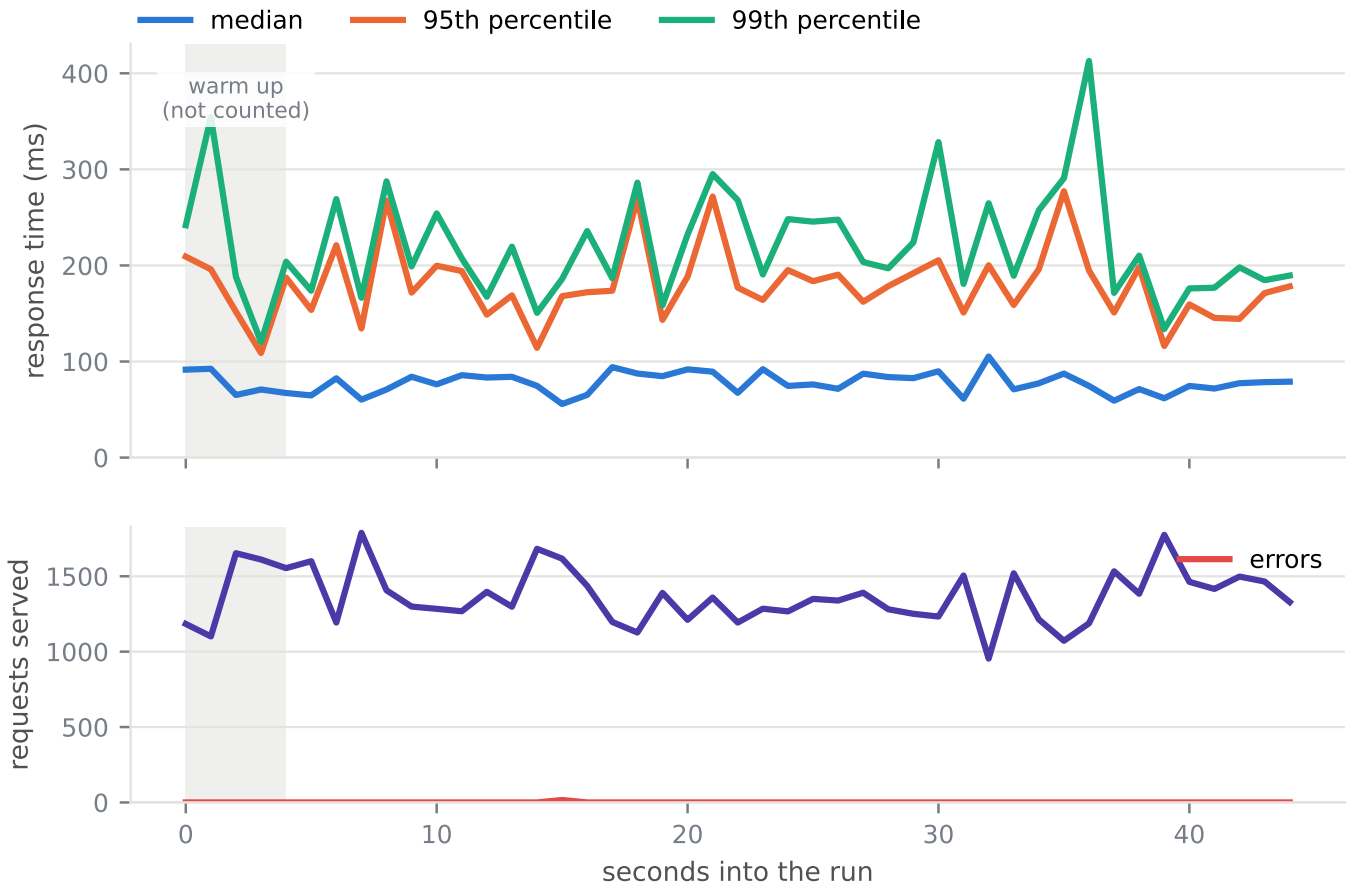
Steady load through the balancer, one back end stopped a third of the way in and started again two thirds of the way in, and no client told about any of it.

Requests completed	35,859
Throughput across the whole run	639 per second
Response time, median / 95th	176 ms / 361 ms
Requests the clients saw fail	64
Requests the balancer replayed onto another back end	22
Messages checked in /feed afterwards	30,366 sent, 1856 missing, 0 duplicated

Load score per backend, and where the traffic went



Response time while a backend is killed and restarted



```

2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:40908->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:40974->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 172.17.0.32:4000 marked DOWN after 3 failures: read tcp 172.17.0.30:41144-
>172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:41144->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:41026->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:51050->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:41304->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:41748->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:41956->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:41928->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:41816->172.17.0.32:4000: read: connection reset by peer
2026/09/13 17:02:07 retrying POST /message on 172.17.0.31:4000 after 172.17.0.32:4000 failed: read tcp
172.17.0.30:41668->172.17.0.32:4000: read: connection reset by peer

```

What the balancer logged.

10 What limits it

SYSTEM	MEAN CPU	WHAT IS ON IT
Sys1	44%	load balancer, and another project's service
Sys2	45%	chat-1
Sys3	45%	chat-2
Sys4	69%	chat-3 and MongoDB

Each of these containers has one core, so a mean near 100% means that machine is the ceiling. Sys4 is the ceiling here, and it is the ceiling for the whole cluster rather than for its own back end: it holds the database that the other two write to. The two processes' own CPU counters say where that core goes.

```

### Sys4 CPU split under load, 2026-09-09 08:46 UTC ###
measured over 20s with 150 users against the balancer

chat-3 (Node)    0.13 cores
mongod           0.80 cores
together         0.93 of the 1.00 core this container has

```

Sys4 under load, from `sources/ops/cpu-split.sh`. Every message from all three back ends is an insert into that one process on that one core.

Which is why the balancer routes away from Sys4 under load rather than sharing equally, and why round robin is so much slower: work sent to Sys4 competes with the database that the other two machines are waiting on, so an evenly shared request is not an evenly priced one.

The obvious question is whether the database should go back on Sys1 with the balancer. Measured, no: the balancer alone already uses about 44% of Sys1's core at this load, and it handles every request twice, once inbound and once outbound. Adding three quarters of a core of database to it asks for more than the machine has, and Sys1 is also the container that has already been out-of-memory-killed 57 times with another project's service on it. The database is where it is because that was the least bad of the two places available.

Getting it to stay there took two changes. MongoDB sizes its cache from the machine it can see rather than the cgroup it is in, and its smallest accepted setting for that, 0.25 GB, was still enough to grow the process to 442 MB of a 512 MB

container and have the kernel kill something mid-run. The cache is pinned to 112 MB through the engine configuration string, which WiredTiger reads after the value the documented flag produces, and the back ends were cut from sixteen connections each to six. A three minute run at this load went from 8,064 failed requests to none.

11 Running it

```
# deploy everything, or restart it after a container reboot
sources/ops/deploy.sh all

# just one part of it
sources/ops/deploy.sh db | backends | lb

# one measured run: load, per-second response times, and CPU and memory
# for all four systems
sources/ops/experiment.sh headline 60 -users 150 -verify

# kill a back end in the middle of a run and watch the balancer notice
sources/ops/failover.sh 60

# how Sys4's one core is divided between the back end and the database
sources/ops/cpu-split.sh 20

# every routing policy and threshold, three runs each
sources/ops/threshold-sweep.sh 3 30

# capture the transcripts the report quotes
sources/ops/evidence.sh

# rebuild this report from whatever is in results/
python3 report/plots.py headline && python3 report/build.py headline
```

Deployment and experiments. The deploy script is idempotent, so it is also the way back after a container restart.

sources/lb/main.go	the load balancer
sources/loadgen/main.go	the load generator
sources/server/	the chat back end, as deployed
server.js	startup and wiring
src/api/router.js	/message and /feed
src/api/feedStore.js	the in-memory room /feed is served from
src/api/writeQueue.js	batched, idempotent writes
src/api/loadMetrics.js	what a back end reports about its load
src/db/	connection, indexes, message storage
src/routes/health.js	/health and /lb/load
sources/ops/	deploy, supervise, stop, experiment, sweep,
	failover, cpu-split, evidence, sample-system
sources/tools/	replica set admin, migration, cleanup
sources/cross_replica_test.js	the browser chat, across all three back ends
sources/assignment1/	the previous assignment's sources
results/	every measurement this report quotes
results/assignment1/	the previous assignment's results, untouched
report/	template, stylesheet, figures, build scripts

What is where in the repository.

12 The load balancer

```
// Performance-based load balancer for Sys1, in front of the chat backends on
// Sys2, Sys3 and Sys4.
//
// The routing decision is not a rotation. Every backend reports how loaded it
// is several times a second, the balancer turns those reports into one score
// per backend, and a backend whose score climbs past a threshold stops
// receiving new traffic until it recovers. Among the backends still under the
// threshold, each request goes to the least loaded one.
//
// Why a threshold at all, rather than simply always picking the minimum: the
// scores are sampled, so between two samples every request would see the same
// minimum and pile onto one machine. The threshold makes the common case cheap
// and stable, and turns the interesting case into an explicit, tunable
// decision that can be measured and reported on.
//
// Three things feed the score, because no single one is sufficient:
//
// CPU          the container's own CPU use, read from its cgroup. The headline
//              measure of system load.
// loop lag     how far behind the backend's event loop is. A Node process
//              waiting on the database has low CPU and a growing queue, and
//              this is what notices.
// latency      what the balancer itself has recently measured for the backend,
//              which catches everything the backend does not report.
//
// Plus, per request, the number of requests already in flight to that backend.
// That one is exact and instantaneous, and it is what stops a burst of
// arrivals between two samples from herding onto the same machine.
package main

import (
    "bufio"
    "bytes"
    "context"
    crand "crypto/rand"
    "crypto/tls"
    "encoding/hex"
    "encoding/json"
    "errors"
    "flag"
    "fmt"
    "io"
    "log"
    "math"
    "math/rand/v2"
    "net"
    "net/http"
    "net/http/httputil"
    "net/url"
    "os"
    "runtime"
    "runtime/debug"
    "sort"
    "strconv"
    "strings"
    "sync"
    "sync/atomic"
    "syscall"
    "time"
)

// -----
// Tuning
// -----

// Reference points that turn a raw measurement into a 0..1 term. Each is the
// value at which that signal is considered to be contributing "fully loaded".
const (
    lagReference      = 60.0 // ms of event loop delay
    latencyReference  = 250.0 // ms of round trip measured at the balancer
    queueReference    = 200.0 // documents waiting to be written
    inflightReference = 96.0  // requests in flight to one backend
)

// Weights of the polled signals. They sum to 1, so a score is directly
// comparable with the threshold.
```

```

const (
    weightCPU      = 0.50
    weightLag      = 0.20
    weightLatency  = 0.15
    weightQueue    = 0.15
)

// How much of the per-request score comes from the instantaneous in-flight
// count rather than the polled snapshot.
const inflightShare = 0.25

// New samples are blended into the previous score rather than replacing it, so
// one unlucky probe does not move traffic on its own.
const scoreSmoothing = 0.45

// -----
// Backends
// -----

type Backend struct {
    ID      int
    URL     *url.URL
    Host    string
    Proxy   *httputil.ReverseProxy

    // Hot path, read and written per request.
    inflight  atomic.Int64
    served    atomic.Int64
    stickyHits atomic.Int64
    failures  atomic.Int64
    latencyEWMA atomic.Uint64 // microseconds, fixed point

    mu      sync.RWMutex
    alive   bool
    overloaded bool
    okStreak int
    failStreak int
    score    float64
    probe    probeResult
}

// What a backend reports about itself.
type probeResult struct {
    CPU      float64 `json:"cpu"`
    ProcCPU  float64 `json:"procCpu"`
    LoopLagMs float64 `json:"loopLagMs"`
    Inflight int    `json:"inflight"`
    QueueDepth int    `json:"queueDepth"`
    FeedSize int64  `json:"feedSize"`
    MemoryFraction float64 `json:"memoryFraction"`
    RSSMB      float64 `json:"rssMB"`
    Backend    string `json:"backend"`

    At time.Time `json:"- "`
    Error string  `json:"- "`
}

func (b *Backend) snapshot() (float64, bool, bool, probeResult) {
    b.mu.RLock()
    defer b.mu.RUnlock()
    return b.score, b.alive, b.overloaded, b.probe
}

// The score used to decide where one request goes. The polled part moves at
// the probe interval; the in-flight part moves with every request, which is
// what spreads a burst that arrives between two probes.
func (b *Backend) instantScore() float64 {
    b.mu.RLock()
    polled := b.score
    b.mu.RUnlock()

    load := float64(b.inflight.Load()) / inflightReference

    return (1-inflightShare)*polled + inflightShare*clamp01(load)
}

func (b *Backend) recordLatency(d time.Duration) {
    const alpha = 0.2

    sample := float64(d.Microseconds())
    previous := math.Float64frombits(b.latencyEWMA.Load())

    next := sample

```

```

    if previous > 0 {
        next = alpha*sample + (1-alpha)*previous
    }

    b.latencyEWMA.Store(math.Float64bits(next))
}

func (b *Backend) latencyMs() float64 {
    return math.Float64frombits(b.latencyEWMA.Load()) / 1000
}

// -----
// Shared state
// -----

var (
    backends []*Backend

    threshold float64
    hysteresis float64
    sticky     bool
    policy     string

    roundRobin atomic.Uint64

    requests  atomic.Int64
    succeeded  atomic.Int64
    failures  atomic.Int64
    retried   atomic.Int64
    abandoned atomic.Int64
    switches  atomic.Int64

    samples latencyRing
    timeline timelineRing

    idPrefix string
    idCounter atomic.Uint64

    memoryLimit int64
)

const stickyCookie = "lb_backend"

const (
    policyPerformance = "performance"
    policyRoundRobin  = "roundrobin"
    policyLeastLoad   = "least-load"
)

// -----
// Selection
// -----

// Picks the backend for one request.
//
// Two stages, in this order.
//
// First the threshold. A backend whose score has climbed past it is set aside
// and receives nothing until it comes back down. If that leaves nothing at
// all, every backend is busy and the request still has to go somewhere, so the
// least loaded of them is used rather than refusing to serve.
//
// Then, among the backends still under the threshold, a weighted choice on how
// much headroom each one has left. A backend at score 0.1 against a threshold
// of 0.7 has six times the headroom of one at 0.6 and receives roughly six
// times as much traffic.
//
// Weighting rather than simply taking the minimum, because the minimum starves
// machines that are merely second best. One of these three containers also
// hosts the database, so its CPU is permanently a little higher than the other
// two: under a strict minimum it received no traffic at all while it still had
// most of a core spare. Sharing in proportion to headroom uses that machine for
// what it can still do, and the proportion shifts on its own as load moves.
//
// The headroom is computed from the per-request score, so it includes the
// in-flight count. That is what keeps a burst arriving between two probes from
// piling onto whichever backend happened to look best when the last probe ran.
func pick(exclude int) *Backend {
    const maxBackends = 16

    var (
        pool    [maxBackends]*Backend
        weights [maxBackends]float64
    )

```

```

    fallback [maxBackends]*Backend
)

poolCount, fallbackCount := 0, 0

for _, b := range backends {
    if b.ID == exclude {
        continue
    }

    b.mu.RLock()
    up, over := b.alive, b.overloaded
    b.mu.RUnlock()

    if !up || fallbackCount >= maxBackends {
        continue
    }

    fallback[fallbackCount] = b
    fallbackCount++

    // The threshold belongs to the performance policy. The two comparison
    // policies are supposed to be what they are named, so neither of them
    // gets to quietly benefit from it.
    if !over || policy != policyPerformance {
        pool[poolCount] = b
        poolCount++
    }
}

// Everything is over the threshold. Fall back to the least loaded, because
// dropping the request would be worse than sending it to a busy machine.
if poolCount == 0 {
    if fallbackCount == 0 {
        return nil
    }

    best := fallback[0]
    bestScore := best.instantScore()

    for i := 1; i < fallbackCount; i++ {
        if score := fallback[i].instantScore(); score < bestScore {
            best, bestScore = fallback[i], score
        }
    }

    return best
}

if poolCount == 1 {
    return pool[0]
}

// The two comparison policies exist so the report can put numbers against
// the claim that the performance-based one is worth having, rather than
// asserting it. Neither is the deployed default.
switch policy {
case policyRoundRobin:
    // A fixed rotation. It ignores how loaded each backend is, which is
    // exactly what makes it the baseline rather than the answer.
    return pool[int(roundRobin.Add(1)%uint64(poolCount))]

case policyLeastLoad:
    // Always the lowest score, with no threshold and no weighting. Useful
    // for showing what the threshold is actually buying.
    best := pool[0]
    bestScore := best.instantScore()
    for i := 1; i < poolCount; i++ {
        if score := pool[i].instantScore(); score < bestScore {
            best, bestScore = pool[i], score
        }
    }
    return best
}

// A backend sitting right on the threshold keeps a small share rather than
// dropping to nothing: it has not crossed the line, and a share of zero
// either side of one probe is how a backend ends up flapping.
const minWeight = 0.02

total := 0.0
for i := 0; i < poolCount; i++ {
    headroom := threshold - pool[i].instantScore()

```

```

        if headroom < minWeight {
            headroom = minWeight
        }
        weights[i] = headroom
        total += headroom
    }

    point := rand.Float64() * total
    for i := 0; i < poolCount; i++ {
        point -= weights[i]
        if point <= 0 {
            return pool[i]
        }
    }

    return pool[poolCount-1]
}

// The cookie only governs Socket.IO. Its polling handshake is several requests
// that only the process holding the session can answer, so those have to stay
// together. The two HTTP routes are stateless and are never pinned, which is
// what lets the balancer keep moving them towards whichever backend is idle.
func stickyBackend(r *http.Request) *Backend {
    if !sticky || !isSocketIO(r.URL.Path) {
        return nil
    }

    cookie, err := r.Cookie(stickyCookie)
    if err != nil {
        return nil
    }

    id, err := strconv.Atoi(cookie.Value)
    if err != nil || id < 0 || id >= len(backends) {
        return nil
    }

    b := backends[id]

    b.mu.RLock()
    up := b.alive
    b.mu.RUnlock()

    if !up {
        return nil
    }

    b.stickyHits.Add(1)
    return b
}

func isSocketIO(path string) bool {
    return strings.HasPrefix(path, "/socket.io/")
}

// -----
// Proxying
// -----

// Wraps the real ResponseWriter for one attempt at one backend.
//
// Its job is to know whether anything has reached the client yet. The reverse
// proxy calls ErrorHandler before writing anything when a round trip fails, so
// if that happens and nothing has been written, the request can still be sent
// somewhere else and the client never learns that the first backend was tried.
type attempt struct {
    http.ResponseWriter
    code      int
    wrote     bool
    failed    bool
    abandoned bool
    failure   error
}

func (a *attempt) WriteHeader(code int) {
    a.wrote = true
    a.code = code
    a.ResponseWriter.WriteHeader(code)
}

func (a *attempt) Write(p []byte) (int, error) {
    if !a.wrote {
        a.wrote = true
    }

```

```

    a.code = http.StatusOK
}
return a.ResponseWriter.Write(p)
}

// Needed for the WebSocket upgrade that Socket.IO uses once it is connected.
func (a *attempt) Hijack() (net.Conn, *bufio.ReadWriter, error) {
    hijacker, ok := a.ResponseWriter.(http.Hijacker)
    if !ok {
        return nil, nil, fmt.Errorf("connection cannot be hijacked")
    }
    return hijacker.Hijack()
}

func (a *attempt) Flush() {
    if flusher, ok := a.ResponseWriter.(http.Flusher); ok {
        flusher.Flush()
    }
}

// Requests larger than this are forwarded straight through and not retried,
// because holding an arbitrary body in memory to be able to replay it is a
// better way to run out of memory than to survive a failure.
const maxReplayBytes = 1 << 16

func proxyHandler(w http.ResponseWriter, r *http.Request) {
    start := time.Now()
    requests.Add(1)

    // A message needs an id before it is forwarded, not after. If this request
    // has to be replayed onto a second backend, both copies carry the same id
    // and the database's unique index turns them into one stored message. An
    // id the caller supplied in the body or query string still wins: the
    // backend reads this header only when there is nothing better.
    if strings.HasPrefix(r.URL.Path, "/message") && r.Header.Get("X-Message-Id") == "" {
        r.Header.Set("X-Message-Id", nextMessageID())
    }

    upgrade := isUpgrade(r)

    // Buffer a small body so a failed attempt can be replayed. An upgrade is
    // never replayed, so it is never buffered.
    var body []byte
    replayable := true

    if r.Body != nil && !upgrade {
        if r.ContentLength > maxReplayBytes {
            replayable = false
        } else {
            limited := io.LimitReader(r.Body, maxReplayBytes+1)
            buffered, err := io.ReadAll(limited)
            if err != nil {
                http.Error(w, "could not read request body", http.StatusBadRequest)
                failures.Add(1)
                return
            }
            if len(buffered) > maxReplayBytes {
                replayable = false
            }
            body = buffered
            r.Body = io.NopCloser(bytes.NewReader(body))
        }
    }

    pinned := stickyBackend(r)
    b := pinned
    if b == nil {
        b = pick(-1)
    }

    if b == nil {
        failures.Add(1)
        http.Error(w, "no backend available", http.StatusServiceUnavailable)
        return
    }

    if sticky && pinned == nil && isSocketIO(r.URL.Path) {
        http.SetCookie(w, &http.Cookie{
            Name:    stickyCookie,
            Value:    strconv.Itoa(b.ID),
            Path:    "/",
            MaxAge:   3600,
        })
    }
}

```

```

}

first := serveOnce(w, r, b, body)

// One retry, on a different backend, only when the first attempt failed
// before anything reached the client.
if first.failed && !first.abandoned && !first.wrote && replayable && !upgrade {
    if next := pick(b.ID); next != nil {
        retried.Add(1)
        log.Printf("retrying %s %s on %s after %s failed: %v",
            r.Method, r.URL.Path, next.Host, b.Host, first.failure)

        second := serveOnce(w, r, next, body)
        finish(start, second)
        return
    }
}

finish(start, first)
}

// Whether a failed attempt failed because the client gave up rather than
// because the backend did.
//
// A caller that times out cancels its side of the round trip, and the proxy
// reports that cancellation the way it reports any other error. It is not the
// same thing at all: the backend may be answering perfectly and simply have
// been abandoned mid-answer. Blaming it is actively harmful, because under
// load clients time out in crowds – enough of them at once to push all three
// backends past the failure threshold within the same second and have the
// balancer start refusing traffic with "no backend available" while every
// backend is healthy. That is a load balancer manufacturing an outage out of
// impatience, and it was visible in a 300-user run on 12 September: three
// backends marked down in one second, all of them answering probes normally
// three seconds later.
//
// So: not counted against the backend, and not retried. A retry would send a
// second copy of the request to a second machine on behalf of a client that
// is no longer listening to either.
func clientGaveUp(r *http.Request, err error) bool {
    if errors.Is(err, context.Canceled) {
        return true
    }

    return r.Context().Err() != nil
}

func serveOnce(w http.ResponseWriter, r *http.Request, b *Backend, body []byte) *attempt {
    if body != nil {
        r.Body = io.NopCloser(bytes.NewReader(body))
    }

    b.served.Add(1)

    // Deferred, not written after ServeHTTP returns, because ServeHTTP does not
    // always return. When the client's connection breaks while the response is
    // being copied to it, ReverseProxy panics with http.ErrAbortHandler, which
    // net/http recovers at the top of the connection's goroutine - past this
    // function, so a plain decrement here is simply skipped.
    //
    // That turns the counter into a leak, and the counter is a routing input:
    // instantScore() reads it as a fraction of inflightReference and clamps at
    // one, so a balancer that has seen enough aborted clients believes every
    // backend is permanently at full occupancy. On 14 September, idle, this
    // balancer reported 6376, 6124 and 2781 requests in flight while all three
    // backends reported one each and the process held fourteen goroutines. The
    // instantaneous term had been pinned at its maximum for every backend for
    // hours, which does not take traffic anywhere wrong so much as it stops the
    // term from distinguishing between backends at all: every score was shifted
    // up by the same 0.25, the headroom weights collapsed towards each other,
    // and the per-request spreading that is supposed to catch a burst arriving
    // between two probes had quietly become a uniform random choice.
    //
    // Aborted clients are not rare here, they are what a load generator with a
    // timeout does to every request it gives up on: more than half of the
    // requests this balancer had served were in that state.
    b.inflight.Add(1)
    defer b.inflight.Add(-1)

    sent := time.Now()
    a := &attempt{ResponseWriter: w, code: http.StatusOK}
    b.Proxy.ServeHTTP(a, r)

```

```

    if a.failed {
        if clientGaveUp(r, a.failure) {
            a.abandoned = true
            abandoned.Add(1)
            return a
        }

        b.failures.Add(1)
        markFailure(b, a.failure)
        return a
    }

    // Socket.IO connections stay open for minutes, so timing them would swamp
    // the average with numbers that say nothing about how fast the backend is.
    if !isSocketIO(r.URL.Path) {
        b.recordLatency(time.Since(sent))
    }

    return a
}

func finish(start time.Time, a *attempt) {
    elapsed := time.Since(start)

    if a.failed {
        failures.Add(1)
        if !a.wrote {
            http.Error(a.ResponseWriter, "backend unavailable", http.StatusBadGateway)
        }
        return
    }

    if a.code >= 500 {
        failures.Add(1)
    } else {
        succeeded.Add(1)
    }

    samples.add(elapsed)
}

func isUpgrade(r *http.Request) bool {
    for _, value := range r.Header.Values("Connection") {
        if strings.Contains(strings.ToLower(value), "upgrade") {
            return true
        }
    }
    return false
}

// Message ids are generated here rather than read from a random source per
// request: a counter behind a per-process random prefix is unique across
// restarts and across the three backends, and costs nothing.
func nextMessageID() string {
    return idPrefix + strconv.FormatUint(idCounter.Add(1), 36)
}

// -----
// Probing and scoring
// -----

// Marks a backend down after enough consecutive failures. Called both by the
// prober and by a failed proxy attempt, so a backend that stops answering is
// noticed by real traffic rather than only at the next probe.
func markFailure(b *Backend, err error) {
    b.mu.Lock()
    defer b.mu.Unlock()

    b.okStreak = 0
    b.failStreak++

    if b.alive && b.failStreak >= failureThreshold {
        b.alive = false
        log.Printf("%s marked DOWN after %d failures: %v", b.Host, b.failStreak, err)
    }
}

const (
    failureThreshold = 3
    recoveryThreshold = 2
)

func probeLoop(client *http.Client, path string, every time.Duration) {

```

```

ticker := time.NewTicker(every)
defer ticker.Stop()

for range ticker.C {
    var wg sync.WaitGroup

    for _, b := range backends {
        wg.Add(1)
        go func(b *Backend) {
            defer wg.Done()
            probe(client, b, path)
        }(b)
    }

    wg.Wait()
    timeline.add()
}

func probe(client *http.Client, b *Backend, path string) {
    result, err := fetchLoad(client, b.URL.String()+path)

    b.mu.Lock()
    defer b.mu.Unlock()

    if err != nil {
        b.okStreak = 0
        b.failStreak++
        b.probe.Error = err.Error()
        b.probe.At = time.Now()

        if b.alive && b.failStreak >= failureThreshold {
            b.alive = false
            log.Printf("%s marked DOWN after %d failed probes: %v", b.Host, b.failStreak, err)
        }
        return
    }

    b.failStreak = 0
    b.okStreak++
    b.probe = result
    b.probe.At = time.Now()

    if !b.alive && b.okStreak >= recoveryThreshold {
        b.alive = true
        b.overloaded = false
        log.Printf("%s is back UP after %d good probes", b.Host, b.okStreak)
    }

    raw := weightCPU*clamp01(result.CPU) +
        weightLag*clamp01(result.LoopLagMs/lagReference) +
        weightLatency*clamp01(b.latencyMs()/latencyReference) +
        weightQueue*clamp01(float64(result.QueueDepth)/queueReference)

    if b.score == 0 {
        b.score = raw
    } else {
        b.score = scoreSmoothing*raw + (1-scoreSmoothing)*b.score
    }

    // Crossing the threshold takes traffic away; coming back takes a clear
    // margin, so a backend hovering at the line does not flip on every probe.
    switch {
    case !b.overloaded && b.score > threshold:
        b.overloaded = true
        switches.Add(1)
        log.Printf("%s over threshold (score %.3f > %.2f), routing away", b.Host, b.score, threshold)
    case b.overloaded && b.score < threshold-hysteresis:
        b.overloaded = false
        log.Printf("%s back under threshold (score %.3f), routing to it again", b.Host, b.score)
    }
}

func fetchLoad(client *http.Client, endpoint string) (probeResult, error) {
    var result probeResult

    resp, err := client.Get(endpoint)
    if err != nil {
        return result, err
    }
    defer resp.Body.Close()

    if resp.StatusCode >= 400 {

```

```

        io.Copy(io.Discard, resp.Body)
        return result, fmt.Errorf("status %d", resp.StatusCode)
    }

    if err := json.NewDecoder(resp.Body).Decode(&result); err != nil {
        return result, err
    }

    return result, nil
}

func clamp01(v float64) float64 {
    if math.IsNaN(v) || v < 0 {
        return 0
    }
    if v > 1 {
        return 1
    }
    return v
}

// -----
// Measurement
// -----

// A bounded ring of recent request latencies. Bounded because the balancer is
// meant to run for days: the previous version kept every latency in a slice,
// which is a memory leak with a report attached.
type latencyRing struct {
    mu      sync.Mutex
    values  []int32 // microseconds
    next    int
    full    bool
}

const latencyRingSize = 200_000

func (r *latencyRing) add(d time.Duration) {
    micros := d.Microseconds()
    if micros > math.MaxInt32 {
        micros = math.MaxInt32
    }

    r.mu.Lock()
    if r.values == nil {
        r.values = make([]int32, latencyRingSize)
    }

    r.values[r.next] = int32(micros)
    r.next++
    if r.next == len(r.values) {
        r.next = 0
        r.full = true
    }
    r.mu.Unlock()
}

func (r *latencyRing) sorted() []int32 {
    r.mu.Lock()
    count := r.next
    if r.full {
        count = len(r.values)
    }
    out := make([]int32, count)
    copy(out, r.values[:count])
    r.mu.Unlock()

    sort.Slice(out, func(i, j int) bool { return out[i] < out[j] })
    return out
}

func (r *latencyRing) reset() {
    r.mu.Lock()
    r.next, r.full = 0, false
    r.mu.Unlock()
}

func percentile(sorted []int32, p float64) float64 {
    if len(sorted) == 0 {
        return 0
    }

    index := int(p / 100 * float64(len(sorted)))

```

```

    if index >= len(sorted) {
        index = len(sorted) - 1
    }

    return float64(sorted[index]) / 1000
}

// One row per probe sweep: what every backend looked like and where traffic
// was going. This is what the report's routing plots are drawn from.
type timelineSample struct {
    T          float64 `json:"t"`
    Score      []float64 `json:"score"`
    CPU        []float64 `json:"cpu"`
    LoopLag    []float64 `json:"loopLag"`
    Served     []int64  `json:"served"`
    Inflight   []int64  `json:"inflight"`
    Over       []bool   `json:"over"`
    Alive      []bool   `json:"alive"`
}

type timelineRing struct {
    mu      sync.Mutex
    samples []timelineSample
    started time.Time
}

const timelineLimit = 20_000

func (t *timelineRing) add() {
    sample := timelineSample{T: time.Since(t.started).Seconds()}

    for _, b := range backends {
        score, alive, over, p := b.snapshot()
        sample.Score = append(sample.Score, round(score, 4))
        sample.CPU = append(sample.CPU, round(p.CPU, 4))
        sample.LoopLag = append(sample.LoopLag, round(p.LoopLagMs, 2))
        sample.Served = append(sample.Served, b.served.Load())
        sample.Inflight = append(sample.Inflight, b.inflight.Load())
        sample.Over = append(sample.Over, over)
        sample.Alive = append(sample.Alive, alive)
    }

    t.mu.Lock()
    if len(t.samples) >= timelineLimit {
        t.samples = t.samples[len(t.samples)/2:]
    }
    t.samples = append(t.samples, sample)
    t.mu.Unlock()
}

func (t *timelineRing) snapshot() []timelineSample {
    t.mu.Lock()
    defer t.mu.Unlock()

    out := make([]timelineSample, len(t.samples))
    copy(out, t.samples)
    return out
}

func (t *timelineRing) reset() {
    t.mu.Lock()
    t.samples = nil
    t.started = time.Now()
    t.mu.Unlock()
}

func round(v float64, places int) float64 {
    factor := math.Pow(10, float64(places))
    return math.Round(v*factor) / factor
}

// -----
// Admin endpoints
// -----

func statusHandler(w http.ResponseWriter, r *http.Request) {
    list := make([]map[string]any, 0, len(backends))

    for _, b := range backends {
        score, alive, over, p := b.snapshot()

        list = append(list, map[string]any{
            "id":      b.ID,

```

```

        "url":          b.URL.String(),
        "name":         p.Backend,
        "alive":        alive,
        "over_threshold": over,
        "score":        round(score, 4),
        "cpu":          round(p.CPU, 4),
        "loop_lag_ms":  p.LoopLagMs,
        "queue_depth":  p.QueueDepth,
        "feed_size":    p.FeedSize,
        "memory_used":  round(p.MemoryFraction, 4),
        "rss_mb":       p.RSSMB,
        "latency_ms":   round(b.latencyMs(), 2),
        "inflight":     b.inflight.Load(),
        "served":       b.served.Load(),
        "sticky":       b.stickyHits.Load(),
        "errors":       b.failures.Load(),
        "last_probe":   p.At.Format(time.RFC3339Nano),
        "probe_error":  p.Error,
    })
}

writeJSON(w, map[string]any{
    "policy":      policy,
    "threshold":   threshold,
    "hysteresis":  hysteresis,
    "sticky_socketio": sticky,
    "switches":    switches.Load(),
    "backends":    list,
})
}

func metricsHandler(w http.ResponseWriter, r *http.Request) {
    sorted := samples.sorted()

    perBackend := map[string]int64{}
    for _, b := range backends {
        perBackend[b.Host] = b.served.Load()
    }

    writeJSON(w, map[string]any{
        "total":      requests.Load(),
        "success":     succeeded.Load(),
        "failed":      failures.Load(),
        "retried":     retried.Load(),
        "abandoned":  abandoned.Load(),
        "switches":    switches.Load(),
        "memory":      memoryStats(),
        "samples":     len(sorted),
        "p50_ms":      round(percentile(sorted, 50), 3),
        "p95_ms":      round(percentile(sorted, 95), 3),
        "p99_ms":      round(percentile(sorted, 99), 3),
        "per_backend": perBackend,
    })
}

// What this process is actually costing, which is not a curiosity here: this
// balancer shares a 512 MB cgroup with another project's service, and when the
// total goes over, the kernel kills the largest thing in it. That was this
// process, repeatedly, before the limits below were set from measurement
// rather than from guesswork.
//
// heap_mb is what Go is holding, sys_mb is what it has taken from the
// operating system and is the number the cgroup counts, and goroutines is
// what concurrency is costing, since each one owns a stack and, for a
// proxied request, connection buffers at both ends.
func memoryStats() map[string]any {
    var m runtime.MemStats
    runtime.ReadMemStats(&m)

    return map[string]any{
        "heap_mb":    round(float64(m.HeapAlloc)/(1<<20), 1),
        "sys_mb":     round(float64(m.Sys)/(1<<20), 1),
        "stacks_mb":  round(float64(m.StackSys)/(1<<20), 1),
        "goroutines": runtime.NumGoroutine(),
        "gc_cycles":  m.NumGC,
        "limit_mb":   memoryLimit >> 20,
    }
}

func timelineHandler(w http.ResponseWriter, r *http.Request) {
    names := make([]string, 0, len(backends))
    for _, b := range backends {
        names = append(names, b.Host)
    }
}

```

```

    }

    writeJSON(w, map[string]any{
        "backends": names,
        "threshold": threshold,
        "samples": timeline.snapshot(),
    })
}

// Clears the counters between experiments.
func resetHandler(w http.ResponseWriter, r *http.Request) {
    requests.Store(0)
    succeeded.Store(0)
    failures.Store(0)
    retried.Store(0)
    switches.Store(0)

    for _, b := range backends {
        b.served.Store(0)
        b.stickyHits.Store(0)
        b.failures.Store(0)
        b.latencyEWMA.Store(0)
    }

    samples.reset()
    timeline.reset()

    w.Write([]byte("reset done\n"))
}

func writeJSON(w http.ResponseWriter, v any) {
    w.Header().Set("Content-Type", "application/json")
    encoder := json.NewEncoder(w)
    encoder.SetIndent("", " ")
    encoder.Encode(v)
}

// -----
// Socket buffers
// -----

// How much kernel memory one connection may hold in each direction.
//
// This is the setting that keeps this container alive, and it took measuring
// to find, because everything about the symptom pointed somewhere else. The
// balancer was being killed under load on a 512 MB cgroup, so the Go heap was
// the obvious suspect; it was not the heap. At thirteen hundred connections
// the heap was 68 MB and the cgroup was at 511 MB, and the difference was
// almost entirely one line of /sys/fs/cgroup/memory.stat:
//
// anon=68MB file=5MB slab=5MB sock=426MB
//
// Socket buffers are charged to the cgroup, and Linux auto-tunes them upward
// per connection with no idea that a limit exists – about 340 KB per socket
// here, which is fine for one connection and fatal for thirteen hundred. The
// kernel then reclaims by killing the largest process in the cgroup, which is
// the balancer, for memory the balancer never allocated.
//
// Setting the buffer sizes explicitly turns auto-tuning off and makes the cost
// per connection a number we choose: 32 KB each way, so two thousand
// connections cost about 128 MB rather than 680 MB. On a local network this
// costs nothing in throughput – 32 KB against a fraction of a millisecond of
// round trip is far more in flight than any of this needs – and what it buys
// is that the ceiling stops moving with the load.
const socketBufferBytes = 32 << 10

// Applied to a socket before it is used, on both the listening sockets (from
// which accepted connections inherit it) and the connections dialled to the
// backends.
func capSocketBuffers(_, _ string, c syscall.RawConn) error {
    var setErr error

    err := c.Control(func(fd uintptr) {
        if err := syscall.SetsockoptInt(int(fd), syscall.SOL_SOCKET, syscall.SO_RCVBUF, socketBufferBytes); err != nil {
            setErr = err
            return
        }

        setErr = syscall.SetsockoptInt(int(fd), syscall.SOL_SOCKET, syscall.SO_SNDBUF, socketBufferBytes)
    })
    if err != nil {
        return err
    }
}

```

```

    return setErr
}

// -----
// Copy buffers
// -----

// ReverseProxy copies a backend response to the client through a 32 KB buffer
// that it allocates per request and then throws away. One buffer is nothing;
// a thousand requests in flight is 32 MB of garbage created and collected
// continuously, on a container with one core to collect it on and 512 MB to
// hold it in. Handing the proxy a pool makes those buffers be reused instead,
// which removes both the footprint and the collector work that came with it.
//
// A pooled buffer is the whole reason /feed stays affordable under load: that
// response is the largest thing this balancer moves, and it is moved through
// one of these rather than through a fresh allocation per reader.
type bufferPool struct {
    pool sync.Pool
}

func newBufferPool(size int) *bufferPool {
    return &bufferPool{
        pool: sync.Pool{
            New: func() any {
                buffer := make([]byte, size)
                return &buffer
            },
        },
    }
}

func (p *bufferPool) Get() []byte {
    return *(p.pool.Get().(*[]byte))
}

func (p *bufferPool) Put(b []byte) {
    p.pool.Put(&b)
}

// -----
// Startup
// -----

// Go sees the host's core count, not the container's share of it. Left alone
// on a 120-core host it would run a 120-way scheduler and garbage collector
// inside a one-core cgroup, and spend a good part of that core on the
// contention that causes.
func applyCPULimit() int {
    raw, err := os.ReadFile("/sys/fs/cgroup/cpu.max")
    if err != nil {
        return runtime.GOMAXPROCS(0)
    }

    fields := strings.Fields(string(raw))
    if len(fields) != 2 || fields[0] == "max" {
        return runtime.GOMAXPROCS(0)
    }

    quota, err1 := strconv.ParseFloat(fields[0], 64)
    period, err2 := strconv.ParseFloat(fields[1], 64)
    if err1 != nil || err2 != nil || period <= 0 {
        return runtime.GOMAXPROCS(0)
    }

    limit := int(math.Ceil(quota / period))
    if limit < 1 {
        limit = 1
    }

    runtime.GOMAXPROCS(limit)
    return limit
}

// The same problem as GOMAXPROCS, one level down, and the one that actually
// took this process out: Go sizes its garbage collector against the memory it
// believes it has. By default it lets the heap grow to twice the live set
// before collecting, and on a host that reports 128 GB it has no reason to
// stop. Inside a 512 MB cgroup shared with another project's service, the
// kernel reaches its limit long before Go reaches its own, and the process is
// killed rather than collected.
//

```

```

// Measured, this is not hypothetical. A grading run on 12 September killed the
// balancer three times in forty seconds; nearly half of that run's requests
// failed, and every backend stayed healthy throughout. The balancer was the
// only thing that died.
//
// A soft memory limit inverts that: as the heap approaches the budget the
// collector runs continuously instead of growing, so the process trades
// throughput for staying alive. It is a ceiling to be approached, not a
// target, which is why the allocation itself is also cut back (the buffer
// pool and the smaller connection buffers below) rather than relying on this
// alone. A limit that the live heap genuinely exceeds only turns an
// out-of-memory kill into a collector spinning on one core.
//
// The share is deliberately well under the cgroup. This container also runs an
// unrelated service of about 150 MB that is not ours to move, plus the page
// cache, and being a good neighbour here is also self-interest: the kernel
// picks its victim by size, and for most of this container's life that was us.
func applyMemoryLimit() int64 {
    const fallbackMB = 192

    budget := int64(fallbackMB) << 20

    if raw := strings.TrimSpace(os.Getenv("LB_MEMORY_MB")); raw != "" {
        if mb, err := strconv.ParseInt(raw, 10, 64); err == nil && mb > 0 {
            debug.SetMemoryLimit(mb << 20)
            return mb << 20
        }
    }

    raw, err := os.ReadFile("/sys/fs/cgroup/memory.max")
    if err == nil {
        text := strings.TrimSpace(string(raw))
        if text != "max" {
            if total, err := strconv.ParseInt(text, 10, 64); err == nil && total > 0 {
                budget = total * 40 / 100
            }
        }
    }

    debug.SetMemoryLimit(budget)
    return budget
}

func main() {
    listenAddr := flag.String("listen", ":4000", "plain HTTP port")
    tlsListen := flag.String("tls-listen", ":3000", "HTTPS port, needs -tls-cert and -tls-key")
    list := flag.String("backends", "", "comma separated backend urls")
    loadPath := flag.String("load-path", "/lb/load", "endpoint on each backend that reports its load")
    probeEvery := flag.Duration("probe-interval", 300*time.Millisecond, "how often to ask each backend how loaded it is")
    thresholdFlag := flag.Float64("threshold", 0.30, "load score above which a backend stops receiving new requests")
    hysteresisFlag := flag.Float64("hysteresis", 0.08, "how far back under the threshold a backend must come to be used again")
    policyFlag := flag.String("policy", policyPerformance,
        "performance (threshold plus headroom weighting), roundrobin, or least-load")
    useSticky := flag.Bool("sticky", true, "pin Socket.IO sessions to one backend with a cookie")
    tlsCert := flag.String("tls-cert", "", "PEM certificate")
    tlsKey := flag.String("tls-key", "", "PEM private key")
    flag.Parse()

    if *list == "" {
        log.Fatalf("give me some backends with -backends")
    }

    procs := applyCPULimit()
    memoryLimit = applyMemoryLimit()
    memoryBudget := memoryLimit

    threshold = *thresholdFlag
    hysteresis = *hysteresisFlag
    sticky = *useSticky
    policy = *policyFlag

    switch policy {
    case policyPerformance, policyRoundRobin, policyLeastLoad:
    default:
        log.Fatalf("unknown policy %q", policy)
    }
    timeline.started = time.Now()

    seed := make([]byte, 6)
    if _, err := crand.Read(seed); err != nil {
        log.Fatalf("could not seed message ids: %v", err)
    }
}

```

```

idPrefix = "lb-" + hex.EncodeToString(seed) + "-"

// One transport, shared by every backend. Sharing it means one connection
// pool and one set of idle connections rather than three, and the default
// of two idle connections per host is what capped the previous version at
// a few hundred requests a second.
//
// The buffer sizes are per connection and are held for as long as the
// connection is idle, so they multiply by the pool size rather than by the
// number of requests: 512 idle connections to each of three backends at
// 32 KB in each direction is 98 MB reserved for buffers alone, which this
// container does not have. 8 KB is comfortably more than a chat message,
// and a /feed response simply arrives in more reads.
//
// MaxConnsPerHost is a cap on how much concurrency may be handed to one
// backend at a time, and the reason to have one is that the backend is a
// single-core Node process. Two thousand callers arriving at once cannot
// be served any faster by opening two thousand sockets to it; all that
// does is move the queue into the backend, where each waiting request also
// costs a socket and a slice of an event loop that has other work to do.
// Past the cap the transport makes the request wait here instead, in a Go
// process with room to wait in.
//
// The number is set by the backends' memory rather than by their speed,
// because every connection this balancer opens is also a socket on the
// far side, with kernel buffers charged to that container's 512 MB. At
// 512 connections each, three backends are being handed fifteen hundred
// sockets; on 13 September that killed chat-1 outright, in the middle of
// a grading run, while it was serving /feed - the largest response it has
// and so the one with the most buffered behind it.
//
// 128 is still far more than a single core can retire: at the hundred
// milliseconds a request actually takes, it is about a thousand requests
// a second per backend, several times what any of them can do. So this
// costs nothing in throughput and removes an entire class of failure.
//
// The idle timeout has to stay below the backends' keep-alive timeout, so
// that the pool closes its connections rather than finding them already
// closed. They are at 120 seconds.
transport := &http.Transport{
    Proxy:           nil,
    MaxIdleConns:    384,
    MaxIdleConnsPerHost: 128,
    MaxConnsPerHost: 128,
    IdleConnTimeout: 90 * time.Second,
    DisableCompression: true,
    ForceAttemptHTTP2: false,
    ExpectContinueTimeout: 0,
    WriteBufferSize: 8 << 10,
    ReadBufferSize: 8 << 10,
    DialContext: (&net.Dialer{
        Timeout: 3 * time.Second,
        KeepAlive: 30 * time.Second,
        Control: capSocketBuffers,
    }).DialContext,
}

copyBuffers := newBufferPool(32 << 10)

for i, raw := range strings.Split(*list, ",") {
    parsed, err := url.Parse(strings.TrimSpace(raw))
    if err != nil {
        log.Fatalf("bad url %s: %v", raw, err)
    }

    b := &Backend{ID: i, URL: parsed, Host: parsed.Host, alive: true}
    b.Proxy = httputil.NewSingleHostReverseProxy(parsed)
    b.Proxy.Transport = transport
    b.Proxy.BufferPool = copyBuffers
    b.Proxy.FlushInterval = -1

    host := parsed.Host
    b.Proxy.ModifyResponse = func(resp *http.Response) error {
        resp.Header.Set("X-LB-Backend", host)
        return nil
    }

    // Reached when the round trip failed, before anything has been written
    // to the client. Recording it on the attempt lets the caller decide
    // whether to try somewhere else.
    b.Proxy.ErrorHandler = func(w http.ResponseWriter, r *http.Request, err error) {
        if a, ok := w.(*attempt); ok {
            a.failed = true

```

```

        a.failure = err
        return
    }
    http.Error(w, "backend error", http.StatusBadGateway)
}

backends = append(backends, b)
log.Printf("backend %d: %s", i, parsed)
}

probeClient := &http.Client{
    Timeout: 2 * time.Second,
    Transport: &http.Transport{
        MaxIdleConnsPerHost: 4,
        IdleConnTimeout:     30 * time.Second,
        DisableCompression: true,
    },
}

go probeLoop(probeClient, *loadPath, *probeEvery)

mux := http.NewServeMux()
mux.HandleFunc("/lb/health", func(w http.ResponseWriter, r *http.Request) {
    w.Write([]byte("ok\n"))
})
mux.HandleFunc("/lb/status", statusHandler)
mux.HandleFunc("/lb/metrics", metricsHandler)
mux.HandleFunc("/lb/timeline", timelineHandler)
mux.HandleFunc("/lb/reset", resetHandler)
mux.HandleFunc("/", proxyHandler)

newServer := func(addr string) *http.Server {
    return &http.Server{
        Addr:      addr,
        Handler:   mux,
        ReadHeaderTimeout: 10 * time.Second,
        IdleTimeout: 120 * time.Second,
        // No write timeout on purpose: a Socket.IO connection is meant to
        // stay open, and a deadline here would cut it off mid-session.
    }
}

log.Printf("policy: %s, threshold %.2f, hysteresis %.2f, probe every %s",
    policy, threshold, hysteresis, *probeEvery)
log.Printf("GOMAXPROCS set to %d from the cgroup CPU limit", procs)
log.Printf("GOMEMLIMIT set to %d MB, the share of the cgroup this process may use", memoryBudget>>20)

errs := make(chan error, 2)

// The listeners are built rather than left to ListenAndServe, because the
// socket buffer size has to be set on the listening socket for accepted
// connections to inherit it. See capSocketBuffers: this is what stops a
// thousand clients from costing more kernel memory than the container has.
listenConfig := &net.ListenConfig{Control: capSocketBuffers}

listen := func(addr string) (net.Listener, error) {
    return listenConfig.Listen(context.Background(), "tcp", addr)
}

if *tlsCert != "" && *tlsKey != "" {
    server := newServer(*tlsListen)
    // HTTP/2 is turned off for this listener. Socket.IO's WebSocket
    // transport is an HTTP/1.1 upgrade, which HTTP/2 has no room for, so
    // leaving h2 on would quietly push every browser client onto the
    // slower long-polling transport.
    server.TLSNextProto = map[string]func(*http.Server, *tls.Conn, http.Handler){}

    ln, err := listen(*tlsListen)
    if err != nil {
        log.Fatalf("could not listen on %s: %v", *tlsListen, err)
    }

    go func() {
        log.Printf("HTTPS listening on %s", *tlsListen)
        errs <- server.ServeTLS(ln, *tlsCert, *tlsKey)
    }()
}

plain, err := listen(*listenAddr)
if err != nil {
    log.Fatalf("could not listen on %s: %v", *listenAddr, err)
}

```

```
go func() {  
    log.Printf("HTTP listening on %s (socket buffers capped at %d KB each way)",  
        *listenAddr, socketBufferBytes>>10)  
    errs <- newServer(*listenAddr).Serve(plain)  
}()  
  
log.Fatal(<-errs)  
}
```

[sources/lb/main.go](#)

13 The two routes, and what they write to

```
'use strict';

// The two HTTP routes the assignment specifies, /message and /feed.
//
// These sit alongside the Socket.IO chat rather than replacing it. Both write
// into the same `messages` collection in the same room, so a message posted
// over HTTP shows up in the browser chat and a message typed in the browser
// shows up in /feed. There is one room and one history, reachable two ways.
//
// What the HTTP route deliberately does not do is invent a signature. The
// Socket.IO path refuses any message it cannot attribute to a key the sender
// proved they hold, and that has not changed. An HTTP caller has no key, so
// its messages are stored unsigned and are reported as unsigned everywhere the
// browser client already reports that. Encryption at rest is unconditional:
// every message stored by either route is AES-256-GCM encrypted first.

const { Router } = require('express');
const { randomUUID } = require('node:crypto');

const { encrypt } = require('../crypto/messageCipher');

const ROOM_ID = 'main';

// Longer than the Socket.IO limit of 500, because this route exists to be
// driven by a load generator sending messages of "random/variable length" and
// silently truncating those would make /feed disagree with what was sent.
const MAX_MESSAGE_BYTES = 16_384;
const MAX_NAME_LENGTH = 64;

// The assignment names these fields exactly, and those spellings are what the
// examples below use. The alternatives are accepted as well because the
// grading load generator is not ours and a request that is obviously a message
// should not be refused over a hyphen.
const NAME_FIELDS = ['client-name', 'client_name', 'clientName', 'clientname', 'name', 'username'];
const MESSAGE_FIELDS = ['msg', 'message', 'text'];
const ID_FIELDS = ['id', 'msgId', 'msg-id', 'message-id', 'messageId'];

function pick(source, fields) {
  if (!source || typeof source !== 'object') return undefined;

  for (const field of fields) {
    const value = source[field];
    if (value !== undefined && value !== null && value !== '') return value;
  }

  return undefined;
}

// Body first, then query string, so either style of client works.
function field(req, fields) {
  return pick(req.body, fields) ?? pick(req.query, fields);
}

function badRequest(res, reason) {
  res.status(400).json({ ok: false, error: reason });
}

/**
 * @param {object} deps
 * @param {import('../feedStore').FeedStore} deps.feed
 * @param {import('../writeQueue').WriteQueue} deps.writeQueue
 * @param {string} deps.instance
 */
function createApiRouter({ feed, writeQueue, instance }) {
  const router = Router();

  async function handleMessage(req, res) {
    const rawName = field(req, NAME_FIELDS);
    const rawMessage = field(req, MESSAGE_FIELDS);

    if (rawName === undefined) return badRequest(res, 'client-name is required.');
```

```

const text = String(rawMessage);
if (!text.trim()) return badRequest(res, 'msg cannot be empty.');
```

if (Buffer.byteLength(text, 'utf8') > MAX_MESSAGE_BYTES) {
 return res.status(413).json({ ok: false, error: 'msg is too long.' });
}

// Where the message id comes from, in order of preference:
//
// 1. the caller, if it supplied one. A client that retries with the same
// id gets one stored message however many times it sends it.
// 2. the X-Message-Id header, which the load balancer stamps on before
// forwarding. That is what makes it safe for the balancer to replay a
// request onto a second backend when the first one fails mid-request:
// both copies carry the same id, so at most one is stored.
// 3. a fresh UUID, for a caller that does neither.

```

const messageId =
  String(field(req, ID_FIELDS) ?? req.get('x-message-id') ?? '').trim() || randomUUID();

const timestamp = new Date();
const { ciphertext, nonce } = encrypt(text);

let duplicate = false;

try {
  ({ duplicate } = await writeQueue.insert({
    msgId: messageId,
    roomId: ROOM_ID,
    senderId: clientName,
    ciphertext,
    nonce,
    signature: null,
    senderPublicKey: null,
    timestamp,
    clientTimestamp: timestamp.getTime(),
    source: 'http',
  }));
} catch (err) {
  console.error('Could not store message:', err.message);
  return res.status(503).json({ ok: false, error: 'Message could not be stored.' });
}

// Added even when the write was a duplicate: the original may have been
// written by a different backend and not yet have reached this one over
// the change stream. The store keys on the message id, so this is a no-op
// if it is already there.
feed.addLocal({ id: messageId, clientName, text, ts: timestamp.getTime() });

res.json({
  ok: true,
  id: messageId,
  'client-name': clientName,
  msg: text,
  ts: timestamp.getTime(),
  duplicate,
  backend: instance,
});
}

router.post('/message', handleMessage);

// A GET that writes is not how this would be designed from scratch. It is
// here because the grading load generator's method is not ours to choose,
// and a message arriving as a query string should be stored rather than
// refused. It is the same handler, with the same idempotency.
router.get('/message', handleMessage);

// Whether this caller said it can read a gzip body. Deliberately a plain
// substring test rather than a full Accept-Encoding parse: the only thing
// that would change under a correct parse is "gzip;q=0", which no real
// client sends, and this runs on every read of the largest response here.
function acceptsGzip(req) {
  const header = req.headers['accept-encoding'];
  return typeof header === 'string' && header.includes('gzip');
}

router.get('/feed', async (req, res) => {
  const limit = Number.parseInt(req.query.limit, 10) || 0;
  const since = Number.parseInt(req.query.since, 10) || 0;

  res.setHeader('Content-Type', 'application/json; charset=utf-8');
  res.setHeader('Cache-Control', 'no-store');
  res.setHeader('X-Feed-Size', feed.size);

```

```

// Vary, because a cache between here and the client would otherwise be
// free to hand the compressed body to a client that cannot read it.
res.setHeader('Vary', 'Accept-Encoding');

let body = null;

if (acceptsGzip(req)) {
  try {
    body = await feed.gzipPayload({ limit, since });
  } catch (err) {
    // Falling through to the uncompressed body is always correct, so a
    // compressor problem costs bandwidth rather than the route.
    console.error('gzip feed failed, sending it uncompressed:', err.message);
  }

  if (body) res.setHeader('Content-Encoding', 'gzip');
}

// Chunks written rather than joined: the store hands back a prefix that is
// shared between every reader, and joining it onto the tail would copy the
// whole room per request, which is the cost both representations are
// shaped to avoid. Content-Length is known up front either way, so the
// response is still a plain sized body rather than a chunked one.
if (!body) body = feed.payload({ limit, since });

const { chunks, length } = body;

res.setHeader('Content-Length', length);

for (let i = 0; i < chunks.length - 1; i += 1) res.write(chunks[i]);

res.end(chunks[chunks.length - 1]);
});

return router;
}

module.exports = { createApiRouter, ROOM_ID, MAX_MESSAGE_BYTES };

```

[sources/server/src/api/router.js](#)

```

'use strict';

// Groups concurrent inserts into one bulk write.
//
// The problem this solves: under load the chat server spends most of its time
// waiting on round trips to MongoDB, one per message. Those round trips are
// almost entirely latency rather than work, so a hundred messages arriving at
// once cost a hundred waits that could have been one.
//
// The queue collects everything that arrives while it is already waiting on
// the database and sends it as a single unordered bulkWrite. That is the whole
// trick, and it needs no timer: a batch is exactly the messages that turned up
// during the previous batch's round trip, so the batch size follows the
// arrival rate on its own. Idle, that is one document and behaves exactly like
// a plain insert; under load the batch grows to whatever arrived during the
// last round trip, and MongoDB sees that many fewer commands.
//
// Deliberately no artificial delay. Waiting a fixed two milliseconds to fill a
// batch would buy the same batching and charge two milliseconds to every
// message sent to an idle server.
//
// The important property is that this does not weaken durability. Each caller
// gets a promise that settles only after the batch its document was in has
// been acknowledged by the database, so a 200 response still means "written",
// not "queued". Nothing is buffered across a crash.
//
// Duplicate keys are a normal outcome, not an error. Two copies of the same
// message id, whether from a client retry or from the load balancer replaying
// a request onto a second backend, must end up as one row. MongoDB rejects the
// second one with error 11000 and the caller is told it was a duplicate.

const DUPLICATE_KEY = 11000;

class WriteQueue {
  /**
   * @param {import('mongodb').Collection} collection
   * @param {{ maxBatch?: number }} [options]
   */
  constructor(collection, { maxBatch = 1000, maxInFlight = 8 } = {}) {
    this.collection = collection;
  }

```

```

this.maxBatch = maxBatch;

// How many bulk writes may be waiting on the database at once. This is the
// dial that trades batch size against queueing: a low number forces bigger
// batches and fewer commands, but makes a burst wait behind the batches
// already in flight, which shows up in the tail of the response times. The
// value here was chosen by measurement, see the report.
this.maxInFlight = maxInFlight;
this.inFlight = 0;

this.pending = [];
this.scheduled = false;

// Reported to the load balancer, which treats a growing queue as load.
this.depth = 0;
this.batches = 0;
this.documents = 0;
this.duplicates = 0;
}

/**
 * Queues one document and resolves once it is safely in the database.
 *
 * @param {object} document
 * @returns {Promise<{ duplicate: boolean }>}
 */
insert(document) {
  return new Promise((resolve, reject) => {
    this.pending.push({ document, resolve, reject });
    this.depth = this.pending.length;

    // A full batch goes immediately. Otherwise the flush is scheduled for the
    // end of this turn of the event loop, which picks up every request that
    // has already been parsed and is waiting. If the database is busy with
    // as many batches as it is allowed, that flush finds nothing to do and
    // the documents stay here until a slot frees up, which is what makes the
    // batches grow under load.
    if (this.pending.length >= this.maxBatch) {
      this.flush();
    } else if (!this.scheduled) {
      this.scheduled = true;
      setImmediate(() => this.flush());
    }
  });
}

async flush() {
  this.scheduled = false;

  if (this.pending.length === 0) return;
  if (this.inFlight >= this.maxInFlight) return;

  let batch;
  if (this.pending.length > this.maxBatch) {
    batch = this.pending.splice(0, this.maxBatch);
  } else {
    batch = this.pending;
    this.pending = [];
  }

  this.depth = this.pending.length;

  this.inFlight += 1;

  const operations = batch.map((item) => ({ insertOne: { document: item.document } }));

  let writeErrors = [];

  try {
    await this.collection.bulkWrite(operations, { ordered: false });
  } catch (err) {
    writeErrors = normaliseWriteErrors(err);

    // No per-document errors means the whole write failed: the database is
    // unreachable, or the connection dropped. Everyone in the batch hears
    // about it.
    if (writeErrors.length === 0) {
      this.inFlight -= 1;
      for (const item of batch) item.reject(err);
      this.drain();
      return;
    }
  }
}

```

```

    this.inFlight -= 1;

    const failedAt = new Map();
    for (const error of writeErrors) failedAt.set(error.index, error);

    this.batches += 1;
    this.documents += batch.length;

    batch.forEach((item, index) => {
      const error = failedAt.get(index);

      if (!error) {
        item.resolve({ duplicate: false });
        return;
      }

      if (error.code === DUPLICATE_KEY) {
        this.duplicates += 1;
        item.resolve({ duplicate: true });
        return;
      }

      item.reject(new Error(error.errmsg || 'write failed'));
    });

    this.drain();
  }

  // A slot has freed up, so send whatever built up while it was busy.
  drain() {
    if (this.pending.length === 0 || this.scheduled) return;
    if (this.inFlight >= this.maxInFlight) return;

    this.scheduled = true;
    setImmediate(() => this.flush());
  }

  stats() {
    return {
      queueDepth: this.depth,
      batches: this.batches,
      documentsWritten: this.documents,
      duplicatesRejected: this.duplicates,
      averageBatch: this.batches ? Math.round((this.documents / this.batches) * 10) / 10 : 0,
      writesInFlight: this.inFlight,
    };
  }
}

// The driver reports per-document failures in slightly different shapes
// depending on how the write failed, so flatten them into one list of
// { index, code, errmsg }.
function normaliseWriteErrors(err) {
  const raw = err && err.writeErrors;
  if (!raw) return [];

  const list = Array.isArray(raw) ? raw : [raw];

  return list.map((entry) => {
    const inner = entry.err || entry;
    return {
      index: inner.index,
      code: inner.code,
      errmsg: inner.errmsg,
    };
  });
}

module.exports = { WriteQueue };

```

[sources/server/src/api/writeQueue.js](#)

```

'use strict';

// The in-memory copy of the room that /feed is served from.
//
// /feed has to return every message, and it is read far more often than it
// changes, so reading and decrypting the whole collection per request would be
// wasteful in a way that gets worse the longer the room lives. Instead each
// backend keeps the room in memory and serves a pre-serialised buffer.

```

```

//
// Three things keep that copy correct:
//
// 1. On startup the room is read once from MongoDB and decrypted.
// 2. A message this backend accepts is added the moment it is written, so a
//    client that posts and immediately reads sees its own message.
// 3. A change stream on the messages collection brings in what the other two
//    backends wrote. The replica set already exists for the Socket.IO
//    adapter, so tailing it costs nothing extra to set up.
//
// A message can arrive by both routes 2 and 3, so every entry is keyed by its
// message id and the second arrival is dropped. That is the same id the
// database uniqueness constraint uses, so the in-memory copy and the stored
// copy agree on what counts as a duplicate.
//
// Entries are kept as finished JSON text rather than objects. A message is
// serialised once when it arrives instead of once per reader, and the store
// holds roughly the bytes it will send rather than a JavaScript object graph
// several times that size, which matters in a 512 MB container.

const { decrypt } = require('../crypto/messageCipher');
const { toBuffer } = require('../db/messageRepository');
const { GzipFeed } = require('../gzipFeed');

// A safety valve, not a feature. Nothing trims the room, so a long enough run
// would grow this store until the container was killed, taking the chat down
// with it. Past this many messages the oldest ones stop being held in memory.
// They are still in the database and nothing is deleted; what is lost is only
// their place in the /feed response.
//
// The number is set by the tightest of the three containers. Sys4 runs a back
// end and the database in the same 512 MB, and measured under load the database
// wants about 300 MB of that. A full store costs the back end about 78 MB on
// top of its 62 MB baseline, which at 60,000 leaves roughly 70 MB spare rather
// than the 10 MB that 100,000 left. Ten megabytes of headroom is not headroom.
const DEFAULT_MAX_ENTRIES = 60_000;

// How far over the cap the store may run before it trims.
//
// This slack is not a detail. Trimming exactly at the cap means taking one
// element off the front of a hundred thousand element array for every message
// that arrives, and taking from the front of an array moves everything behind
// it. Measured, that turned into the slowest thing in the server: throughput
// fell by two thirds once a run had filled the store. Trimming a tenth of the
// store at a time puts that cost on one message in ten thousand instead of on
// every one, and the store still never holds more than a tenth over its cap.
const TRIM_SLACK = 0.1;

// How many messages may arrive after the cached prefix before it is rebuilt.
// Every reader in between serialises this many entries, and the rebuild costs
// one pass over the room, so this trades a bounded per-read cost against how
// often that pass happens. A thousand is a few tens of kilobytes per read.
const REBASE_AFTER = 1024;

// How far behind the database the in-memory room may be before that counts as
// a gap rather than as writes still on their way. Two checks apart, so this is
// a shortfall that has survived one reconciliation interval.
const RECONCILE_SLACK = 25;

// The least time between two whole-room reads triggered by that check.
const RESYNC_COOLDOWN_MS = 120_000;

// How many recovered messages a resync adds per turn of the event loop. Small
// enough that the turn stays in single-digit milliseconds, so that a resync
// never becomes the reason for the next one.
const RESYNC_CHUNK = 512;

// The two constant responses, built once rather than per request.
const EMPTY_FEED = Buffer.from('[]', 'utf8');
const CLOSE_FEED = Buffer.from(']', 'utf8');

class FeedStore {
  constructor(collection, { maxEntries = DEFAULT_MAX_ENTRIES, roomId = 'main' } = {}) {
    this.collection = collection;
    this.maxEntries = maxEntries;
    this.roomId = roomId;

    this.entries = [];
    this.trimAt = Math.ceil(maxEntries * (1 + TRIM_SLACK));

    // Ids run alongside the entries rather than only in the set, so that when
    // the oldest entries are trimmed their ids can be trimmed with them. A set
    // that only ever grew would outlive the messages it was describing and

```

```

// become the leak the trimming was there to prevent.
this.ids = [];
this.seen = new Set();

// The cached /feed response, held as a prefix that is reused rather than
// rebuilt. See payload() for why it is shaped this way.
this.base = null;
this.baseCount = 0;

// The same room, compressed as it is written rather than as it is read.
// See gzipFeed.js: this is what makes the largest response this deployment
// serves affordable to serve often.
this.gzip = new GzipFeed();

this.dropped = 0;
this.fromStream = 0;
this.stream = null;
this.closed = false;

// Where to pick the change stream back up if it breaks. See watch().
this.resumeToken = null;
this.resyncs = 0;
this.streamFailures = 0;
this.lastResyncAt = 0;
}

get size() {
  return this.entries.length;
}

/**
 * Reads the room out of MongoDB. Called once, before the server listens.
 */
async load() {
  // Newest first with a limit, then reversed, so a room larger than the cap
  // gives us the most recent messages rather than the oldest ones.
  //
  // Read through a cursor in batches rather than with toArray. Three back
  // ends restarting together and each asking one small database for a
  // hundred thousand documents in a single reply is a memory spike on both
  // sides of the connection, and that database has half a gigabyte.
  const cursor = this.collection
    .find({ roomId: this.roomId })
    .sort({ _id: -1 })
    .limit(this.maxEntries)
    .batchSize(2000);

  // Each document is decrypted and serialised as it arrives, and only the
  // finished entry is kept. Collecting the documents themselves and walking
  // them afterwards is the obvious way to write this and it holds the whole
  // room twice over: a stored document carries its ciphertext and nonce as
  // Buffers and the driver's own BSON around them, several times the size of
  // the line of JSON it becomes.
  //
  // That transient is why a backend holding 24,545 messages had a resident
  // size of 142 MB with a live heap of 28 MB. The garbage is collectable and
  // the process is in no danger from it on its own; what it does is leave
  // every backend looking like it is using three times the memory it needs,
  // on a machine where the kernel picks what to kill by exactly that number.
  const prepared = [];
  for await (const document of cursor) {
    const entry = this.prepare(document);
    if (entry !== null) prepared.push(entry);
  }

  prepared.reverse();

  for (const entry of prepared) this.push(entry.id, entry.json);

  return this.entries.length;
}

/**
 * Adds a message this backend has just written. The text is already in hand,
 * so nothing is decrypted here.
 *
 * @returns {boolean} false if this id was already known.
 */
addLocal({ id, clientName, text, ts }) {
  return this.push(id, serialise(id, clientName, text, ts, null));
}

/**

```

```

* Adds a stored document, from the startup read or from the change stream.
*/
addDocument(document) {
  const entry = this.prepare(document);
  if (entry === null) return false;

  return this.push(entry.id, entry.json);
}

/**
* Turns a stored document into the line of JSON /feed will return, without
* storing it. Returns null for a document already held.
*/
prepare(document) {
  const id = document.msgId || String(document._id);
  if (this.seen.has(id)) return null;

  const clientName = document.senderId;
  const ts = document.timestamp ? document.timestamp.getTime() : Date.now();

  let text = null;
  let integrity = null;

  try {
    text = decrypt({
      ciphertext: toBuffer(document.ciphertext),
      nonce: toBuffer(document.nonce),
    });
  } catch {
    // The stored bytes no longer match their authentication tag. The message
    // is still listed, because leaving it out would quietly hide the
    // tampering, but its text is withheld and the verdict is reported.
    integrity = 'failed';
  }

  return { id, json: serialise(id, clientName, text, ts, integrity) };
}

push(id, json) {
  if (this.seen.has(id)) return false;

  this.seen.add(id);
  this.ids.push(id);
  this.entries.push(json);

  // The separator belongs to the entry here, because the compressed stream
  // is written once and never revisited: there is no later pass in which a
  // comma could be inserted between two entries.
  this.gzip.append(this.entries.length === 1 ? `[${json}]` : `,${json}`);

  if (this.entries.length > this.trimAt) {
    const overflow = this.entries.length - this.maxEntries;

    for (const dropped of this.ids.splice(0, overflow)) this.seen.delete(dropped);
    this.entries.splice(0, overflow);

    this.dropped += overflow;

    // The cached prefix described entries that have just been dropped off
    // the front, so it no longer describes anything. This is the one case
    // that cannot be handled by appending, and it happens once per trim
    // rather than once per message.
    this.base = null;
    this.baseCount = 0;

    // And for the same reason the compressed stream has to be replaced: it
    // encodes the dropped messages, and there is no taking bytes off the
    // front of a deflate stream.
    //
    // Handed over as work to be done rather than done here. Rebuilding sixty
    // thousand messages inline blocks the event loop for about a second, and
    // a blocked event loop stops this backend reading its change stream,
    // which makes the reconciler re-read the whole room, which trims again.
    // That loop is what took throughput from 513 requests a second to 38 and
    // left the three backends holding three different rooms. See
    // gzipFeed.rebuild().
    this.gzip.rebuild(this.entries.map((entry, i) => (i === 0 ? `[${entry}]` : `,${entry}`)));
  }

  return true;
}

/**

```

```

* The /feed response body, as the chunks to write and their total length.
*
* The obvious implementation caches the finished response and rebuilds it
* whenever a message arrives. That is fine for a room that is read more
* often than it is written, and wrong for this one: under load every read
* is preceded by a write, so the cache never survives to be used twice and
* every reader pays to serialise the entire room. At twenty thousand
* messages that is a two megabyte string built per request, on a backend
* with one core, and it grows with the room.
*
* So the response is not one buffer but two. The first is a prefix holding
* all but the most recent messages, built once and then handed to reader
* after reader untouched; the second is the handful of messages that have
* arrived since, serialised per request. A write no longer invalidates
* anything - it only lengthens the short half. The prefix is rebuilt when
* the short half stops being short, which is once every REBASE_AFTER
* messages rather than once per message.
*
* Writing them as two chunks rather than joining them is the point: joining
* would copy the prefix, which is the cost this exists to avoid.
*/
/**
* The same whole-room response, gzip encoded.
*
* Returns null when the caller asked for a narrowed view, which is not the
* hot path and is not worth a second cache, or when the compressor has been
* turned off by an error. The caller falls back to payload() in both cases.
*/
async gzipPayload({ limit = 0, since = 0 } = {}) {
  if (limit || since) return null;
  if (!this.gzip.enabled) return null;

  return this.gzip.body();
}

payload({ limit = 0, since = 0 } = {}) {
  if (!limit && !since) {
    if (this.entries.length === 0) return { chunks: [EMPTY_FEED], length: EMPTY_FEED.length };

    if (this.base === null || this.entries.length - this.baseCount > REBASE_AFTER) {
      this.base = Buffer.from(`${this.entries.join(',')}`, 'utf8');
      this.baseCount = this.entries.length;
    }

    const tail =
      this.entries.length > this.baseCount
        ? Buffer.from(`${this.entries.slice(this.baseCount).join(',')}`, 'utf8')
        : CLOSE_FEED;

    return { chunks: [this.base, tail], length: this.base.length + tail.length };
  }

  // The narrowed views are for our own load generator and for anyone who
  // does not want the whole room. They are not cached: they are not the hot
  // path, and caching them would need a key per distinct query.
  let selected = this.entries;

  if (since > 0) {
    selected = selected.filter((entry) => timestampOf(entry) > since);
  }
  if (limit > 0 && selected.length > limit) {
    selected = selected.slice(selected.length - limit);
  }

  const body = Buffer.from(`${selected.join(',')}`, 'utf8');

  return { chunks: [body], length: body.length };
}

/**
* Starts tailing the collection for messages written by the other backends.
*
* A change stream can be closed under us by a failover or a network blip, so
* a broken stream is reopened rather than left dead. The store is keyed by
* message id, so replaying an event that was already applied is harmless.
*/
// Chooses how this backend hears about the other two backends' messages.
// 'stream' tails the collection, 'off' does not sync at all. Set through
// FEED_SYNC so the two can be measured against each other on the real
// deployment rather than argued about.
startSync() {
  const mode = String(process.env.FEED_SYNC || 'stream');
  if (mode === 'off') {

```

```

    console.log('Feed sync disabled');
    return;
  }
  this.watch();
  this.startReconciler();
}

/**
 * Notices a feed that has fallen behind the database, and repairs it.
 *
 * The resume token in watch() is meant to make this unnecessary, and it is
 * what actually keeps the feed complete. This is here because the cost of
 * being wrong about that is not symmetric: a backend that quietly serves an
 * incomplete /feed looks entirely healthy from the outside - it answers
 * every request, quickly, with a valid feed that is simply missing things -
 * and there is nothing in a health check that would ever notice. The gap
 * that cost 599 messages went unnoticed for exactly this reason.
 *
 * So the store is compared against the database periodically. A difference
 * has to persist across two checks before it counts, because under load
 * there are always writes in flight that this backend has not been told
 * about yet, and those close by themselves.
 */
startReconciler() {
  const every = Number.parseInt(process.env.FEED_RECONCILE_MS, 10) || 20_000;
  if (every <= 0) return;

  let shortLastTime = 0;

  this.reconciler = setInterval(async () => {
    if (this.closed) return;

    try {
      // Counted rather than estimated, and counted for this room. The
      // estimate is collection metadata rather than a count, and it drifts
      // badly after a bulk delete - 19,267 against an actual 28,369 on this
      // deployment - which as the basis for "is the feed behind?" would mean
      // reading the whole room again and again to close a gap that is not
      // there. The roomId index makes the real count cheap enough to ask
      // for every twenty seconds.
      const stored = await this.collection.countDocuments({ roomId: this.roomId });
      const held = this.entries.length + this.dropped;
      const short = stored - held;

      // Only a gap that is still there on the next look, only one big
      // enough not to be the writes of the last few seconds, and not more
      // often than the cooldown, because a resync reads the whole room and
      // doing that repeatedly under load is a worse problem than the one it
      // is there to fix.
      const sinceLastResync = Date.now() - this.lastResyncAt;

      if (
        short > RECONCILE_SLACK &&
        shortLastTime > RECONCILE_SLACK &&
        sinceLastResync > RESYNC_COOLDOWN_MS
      ) {
        console.error(`Feed is ${short} messages behind the database, resyncing`);
        shortLastTime = 0;
        this.lastResyncAt = Date.now();
        await this.resync();
        return;
      }

      shortLastTime = short;
    } catch (err) {
      console.error('Feed reconciliation check failed:', err.message);
    }
  }, every);

  // Nothing should be kept alive by this timer.
  if (this.reconciler.unref) this.reconciler.unref();
}

watch() {
  if (this.closed) return;

  // Resuming from the last event seen, rather than reopening blind.
  //
  // This is the difference between a stream that breaks and one that loses
  // data. A change stream does break - "connection to mongod closed" shows
  // up in these logs under load - and reopening a change stream without a
  // resume token starts it at the present moment. Everything written while
  // it was down is then never delivered, and because this store is only

```

```

// ever appended to, never reconciled, that gap is permanent: the backend
// serves a /feed that is missing those messages for as long as it runs.
//
// It is not a theoretical gap. After a grading run on 13 September the
// three backends held 37,339, 37,938 and 37,938 messages: one of them had
// dropped 599 messages and stayed exactly 599 behind, and the grader,
// reading /feed through the balancer, scored the run at 96.86% delivered
// for that reason and no other.
//
// A resume token fixes the common case. Where it cannot - a token too old
// for the oplog to still cover - the fallback is to read the room again
// rather than to carry on with a hole in it.
const options = { fullDocument: 'default' };
if (this.resumeToken) options.resumeAfter = this.resumeToken;

try {
  this.stream = this.collection.watch([{$match: { operationType: 'insert' } }], options);
} catch (err) {
  console.error('Feed change stream could not be opened:', err.message);
  setTimeout(() => this.resync(), 1000);
  return;
}

this.stream.on('change', (event) => {
  // Kept even for events that are filtered out below, because the token
  // marks a position in the stream, not a message we wanted.
  this.resumeToken = event._id;

  if (!event.fullDocument) return;
  if (event.fullDocument.roomId !== this.roomId) return;

  if (this.addDocument(event.fullDocument)) this.fromStream += 1;
});

this.stream.on('error', (err) => {
  if (this.closed) return;

  this.streamFailures += 1;
  console.error('Feed change stream failed, reopening:', err.message);

  const stream = this.stream;
  this.stream = null;
  stream.close().catch(() => {});

  // A token the oplog no longer covers cannot be resumed from, and mongo
  // says so with these. Reading the room again is the only way back to a
  // complete feed.
  const stale =
    err.code === 286 || err.codeName === 'ChangeStreamHistoryLost' || err.code === 280;

  setTimeout(() => (stale ? this.resync() : this.watch()), 1000);
});
}

/**
 * Rebuilds the room from the database and starts the stream again.
 *
 * Used when the stream cannot be resumed. The store is keyed by message id
 * and this only adds, so a resync is safe to run at any time; what it costs
 * is one pass over the room, which is why it is the fallback and not the
 * mechanism.
 */
async resync() {
  if (this.closed) return;

  this.resyncs += 1;
  console.log('Feed resyncing from the database');

  try {
    // Start the stream first, so that anything written during the read is
    // caught by the stream rather than missed by both.
    this.resumeToken = null;
    this.watch();

    const before = this.entries.length;
    const cursor = this.collection
      .find({ roomId: this.roomId })
      .sort({ _id: -1 })
      .limit(this.maxEntries)
      .batchSize(2000);

    // Decrypted and serialised as they arrive, for the same reason load()
    // does it that way: holding every document costs several times what

```

```

    // holding every finished entry costs.
    const prepared = [];
    for await (const document of cursor) {
        const entry = this.prepare(document);
        if (entry !== null) prepared.push(entry);
    }

    prepared.reverse();

    // Added in slices with a yield between them, rather than in one loop.
    //
    // A resync of a room this size is tens of thousands of messages and the
    // loop that adds them does not await anything, so it holds the event
    // loop for as long as it takes - seconds. This is the fallback for a
    // backend that has fallen behind, so it runs exactly when the process is
    // least able to afford going deaf: the change stream it is trying to
    // catch up with is not being read while this runs, and neither are the
    // requests. On 14 September that was one of the steps in a loop where a
    // stalled backend resynced, which stalled it, which made it resync.
    let added = 0;
    for (let i = 0; i < prepared.length; i += RESYNC_CHUNK) {
        const end = Math.min(i + RESYNC_CHUNK, prepared.length);
        for (let j = i; j < end; j += 1) {
            if (this.push(prepared[j].id, prepared[j].json)) added += 1;
        }

        if (end < prepared.length) await new Promise((resolve) => setImmediate(resolve));
        if (this.closed) return;
    }

    console.log(`Feed resynced: ${before} -> ${this.entries.length} messages (${added} recovered)`);
} catch (err) {
    console.error('Feed resync failed, will try again:', err.message);
    setTimeout(() => this.resync(), 5000);
}
}

async close() {
    this.closed = true;
    if (this.reconciler) clearInterval(this.reconciler);
    if (this.stream) await this.stream.close().catch(() => {});
}

stats() {
    return {
        feedSize: this.entries.length,
        feedFromStream: this.fromStream,
        feedDropped: this.dropped,
        feedStreamFailures: this.streamFailures,
        feedResyncs: this.resyncs,
    };
}
}

// One message, as it appears in the /feed array.
//
// The two field names the assignment specifies, "client-name" and "msg", are
// used exactly as given, so what goes in through /message is what comes back
// out of /feed.
function serialise(id, clientName, text, ts, integrity) {
    const record = { id, 'client-name': clientName, msg: text, ts };
    if (integrity) record.integrity = integrity;

    return JSON.stringify(record);
}

// Pulls the ts field back out of a serialised entry without parsing the whole
// thing. Only used by the ?since= view.
function timestampOf(entry) {
    const at = entry.lastIndexOf('"ts:');
    if (at === -1) return 0;

    return Number.parseInt(entry.slice(at + 5), 10) || 0;
}

module.exports = { FeedStore, DEFAULT_MAX_ENTRIES };

```

sources/server/src/api/feedStore.js

```
'use strict';
```

```

// What this backend tells the load balancer about how busy it is.
//
// The load balancer needs a number it can compare across three machines, and
// it needs it cheaply, because it asks several times a second. So the numbers
// are sampled here on a timer and the endpoint just hands back the last
// snapshot. Reading them per request would make the measurement part of the
// load it is measuring.
//
// Four signals are collected, because no single one is enough:
//
//   cpu          the whole container's CPU use, from the cgroup. This is the
//                headline "system load" figure. It has to come from the cgroup
//                and not from /proc/stat, because /proc inside these containers
//                shows the host's 120 cores rather than the one core the
//                container is actually limited to.
//   loopLag      how long the event loop is taking to come back round. A Node
//                process waiting on the database sits at low CPU while its
//                queue grows, and only this number notices.
//   inflight     requests accepted but not yet answered. The fastest-moving
//                signal, and the one that reacts before CPU has caught up.
//   memory       the cgroup's memory use. Not part of the routing score, but
//                worth reporting: these containers have 512 MB and have been
//                OOM-killed before.

const fs = require('node:fs');
const v8 = require('node:v8');
const { monitorEventLoopDelay } = require('node:perf_hooks');

const CGROUP = '/sys/fs/cgroup';
const SAMPLE_INTERVAL_MS = 250;

// How often the event loop delay histogram takes a reading. Every reading
// carries the sampling interval itself as a floor, so an idle process reads
// back roughly this figure rather than zero. It is subtracted again below,
// which leaves 0 ms meaning "not behind" and keeps the number comparable
// between backends.
const LOOP_RESOLUTION_MS = 10;

// Reads one number out of a cgroup file, or null if this kernel does not
// expose it. Every reader below tolerates null so the server still runs on a
// machine without cgroup v2.
function readNumber(file) {
  try {
    return Number(fs.readFileSync(`${CGROUP}/${file}`, 'utf8').trim());
  } catch {
    return null;
  }
}

// Cumulative CPU microseconds used by everything in this container.
function readCpuUsage() {
  try {
    const line = fs
      .readFileSync(`${CGROUP}/cpu.stat`, 'utf8')
      .split('\n')
      .find((row) => row.startsWith('usage_usec '));

    return line ? Number(line.slice('usage_usec '.length)) : null;
  } catch {
    return null;
  }
}

// How many cores this container is allowed. "max 100000" means unlimited, in
// which case fall back to the core count the kernel reports.
function readCpuLimit() {
  try {
    const [quota, period] = fs
      .readFileSync(`${CGROUP}/cpu.max`, 'utf8')
      .trim()
      .split(/\s+/);

    if (quota === 'max') return require('node:os').cpus().length;

    return Number(quota) / Number(period);
  } catch {
    return require('node:os').cpus().length;
  }
}

class LoadMetrics {
  constructor(instance) {
    this.instance = instance;
  }
}

```

```

this.inflight = 0;
this.cpuLimit = readCpuLimit() || 1;

// 10 ms is fine grained enough to see a stalled loop without the timer
// itself costing anything noticeable on a single core.
this.loopDelay = monitorEventLoopDelay({ resolution: LOOP_RESOLUTION_MS });
this.loopDelay.enable();

this.lastCpuUsage = readCpuUsage();
this.lastCpuAt = Date.now();
this.lastProcCpu = process.cpuUsage();

this.snapshot = this.build(0, 0);

this.timer = setInterval(() => this.sample(), SAMPLE_INTERVAL_MS);
// Do not hold the process open just to keep measuring it.
this.timer.unref();
}

// Called by the request middleware. Kept as two plain counters because this
// runs on every single request.
requestStarted() {
  this.inflight += 1;
}

requestFinished() {
  this.inflight -= 1;
}

sample() {
  const now = Date.now();
  const elapsedMs = now - this.lastCpuAt;
  if (elapsedMs <= 0) return;

  // Container CPU, as a fraction of the cores this container may use.
  let cpu = 0;
  const usage = readCpuUsage();
  if (usage !== null && this.lastCpuUsage !== null) {
    cpu = (usage - this.lastCpuUsage) / (elapsedMs * 1000 * this.cpuLimit);
  }
  if (usage !== null) this.lastCpuUsage = usage;

  // This process on its own, so the report can separate the chat server from
  // anything else sharing the container.
  const procDelta = process.cpuUsage(this.lastProcCpu);
  this.lastProcCpu = process.cpuUsage();
  const procCpu = (procDelta.user + procDelta.system) / (elapsedMs * 1000 * this.cpuLimit);

  this.lastCpuAt = now;

  this.snapshot = this.build(clamp01(cpu), clamp01(procCpu));

  // The histogram measures since the last reset, so clear it every sample to
  // keep the figure current rather than an average over all of uptime.
  this.loopDelay.reset();
}

build(cpu, procCpu) {
  const memoryCurrent = readNumber('memory.current');
  const memoryMax = readNumber('memory.max');

  return {
    backend: this.instance,
    cpu,
    procCpu,
    loopLagMs: lagAbove(this.loopDelay.mean),
    loopLagP99Ms: lagAbove(this.loopDelay.percentile(99)),
    inflight: this.inflight,
    memoryUsed: memoryCurrent,
    memoryLimit: memoryMax,
    memoryFraction: memoryCurrent && memoryMax ? round(memoryCurrent / memoryMax, 4) : null,
    rssMB: round(process.memoryUsage().rss() / 1048576, 1),

    // Resident size alone cannot say whether this process is near trouble,
    // because most of what V8 holds is garbage it has not had a reason to
    // collect yet. The live figure is what says whether the room actually
    // fits: on 14 September chat-3's resident size was 142 MB with 24,545
    // messages held, which projected past the container's 512 MB before the
    // store's own 60,000 cap - and its live heap was a tenth of that.
    heapMB: round(v8.getHeapStatistics().used_heap_size / 1048576, 1),
    heapLimitMB: round(v8.getHeapStatistics().heap_size_limit / 1048576, 1),
    uptimeSeconds: Math.round(process.uptime()),
  };
};

```

```

}

// The snapshot plus whatever the caller wants to add, such as the write
// queue depth and the size of the feed.
report(extra = {}) {
  return { ...this.snapshot, inflight: this.inflight, ...extra };
}

}

// Turns a raw histogram reading in nanoseconds into milliseconds behind
// schedule, with the sampling interval taken off.
function lagAbove(nanoseconds) {
  const milliseconds = nanoseconds / 1e6 - LOOP_RESOLUTION_MS;
  return milliseconds > 0 ? round(milliseconds, 2) : 0;
}

function clamp01(value) {
  if (!Number.isFinite(value) || value < 0) return 0;
  return value > 1 ? 1 : value;
}

function round(value, places) {
  const factor = 10 ** places;
  return Math.round(value * factor) / factor;
}

module.exports = { LoadMetrics };

```

sources/server/src/api/loadMetrics.js

14 The load generator

```
// Load generator for the chat API behind the load balancer.
//
// It models users rather than requests. Each virtual user is a goroutine that
// waits a random interval, sends a message of random length, and now and then
// reads the feed instead. The three things the assignment asks to vary are all
// flags: how many users, how long the messages are, and how long the gaps
// between them are.
//
// loadgen -url http://10.1.75.53:4229 -users 60 -duration 60s
//
// What it writes out:
//
// -out          one JSON file with the percentiles, throughput and error
//               counts for the whole run
// -timeseries    a CSV with one row per second, which is what the response
//               time plots in the report are drawn from
//
// With -verify it also checks the run for correctness rather than only for
// speed: every message id it sent is looked for in /feed afterwards, and the
// run fails if any is missing or appears twice. That is the duplicate
// suppression requirement, tested from the outside.
package main

import (
    "bytes"
    "crypto/tls"
    "encoding/json"
    "flag"
    "fmt"
    "io"
    "math/rand"
    "net/http"
    "os"
    "sort"
    "strings"
    "sync"
    "time"
)

type record struct {
    at          time.Duration
    latency     time.Duration
    status      int
    backend     string
    feed        bool
    err         bool
}

type feedEntry struct {
    ID string `json:"id"`
}

var words = strings.Fields(`the quick brown fox jumps over a lazy dog while we
deploy three chat backends behind one balancer and measure how long each
request takes under load with messages of many different lengths sent at
uneven intervals by a changing number of users`)

func main() {
    url := flag.String("url", "http://10.1.75.53:4229", "load balancer base url")
    users := flag.Int("users", 40, "number of concurrent virtual users")
    duration := flag.Duration("duration", 60*time.Second, "how long to keep sending")
    minLen := flag.Int("min-len", 20, "shortest message, in characters")
    maxLen := flag.Int("max-len", 400, "longest message, in characters")
    minGap := flag.Duration("min-interval", 20*time.Millisecond, "shortest pause between one user's messages")
    maxGap := flag.Duration("max-interval", 250*time.Millisecond, "longest pause between one user's messages")
    feedRatio := flag.Float64("feed-ratio", 0.2, "fraction of requests that read /feed instead of posting")
    feedLimit := flag.Int("feed-limit", 0, "if set, ask /feed for only this many recent messages")
    timeout := flag.Duration("timeout", 10*time.Second, "per request timeout")
    warmup := flag.Duration("warmup", 3*time.Second, "discard results from this long at the start")
    experiment := flag.String("experiment", "run", "name recorded in the output")
    out := flag.String("out", "", "write the summary to this JSON file")
    series := flag.String("timeseries", "", "write per-second numbers to this CSV file")
    verify := flag.Bool("verify", false, "afterwards, check every message sent appears in /feed exactly once")
    seed := flag.Int64("seed", 1, "random seed, so a run can be repeated")
    compress := flag.Bool("compress", true,
        "accept a gzip-encoded /feed, as an ordinary HTTP client does")
}
```

```

flag.Parse()

if *minLen > *maxLen {
    fmt.Fprintln(os.Stderr, "-min-len must not exceed -max-len")
    os.Exit(1)
}

transport := &http.Transport{
    MaxIdleConns:      4096,
    MaxIdleConnsPerHost: *users * 2,
    IdleConnTimeout:    90 * time.Second,
    // Left on by default, because every HTTP client a grader is likely to
    // use - Go's own, Python requests, curl, a browser - asks for gzip
    // without being told to, and /feed is where that decides the result.
    // The flag is here so the two can be measured against each other
    // rather than assumed.
    DisableCompression: !*compress,
    // The HTTPS listener uses a self-signed certificate. This tool is
    // pointed at a machine whose address is already known, so skipping the
    // check costs nothing that was being relied on.
    TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
}
client := &http.Client{Transport: transport, Timeout: *timeout}

fmt.Printf("%s: %d users against %s for %s\n", *experiment, *users, *url, *duration)
encoding := "identity"
if *compress {
    encoding = "gzip if offered"
}
fmt.Printf(" message length %d-%d chars, interval %s-%s, %.0f%% of requests read the feed, accepting %s\n",
    *minLen, *maxLen, *minGap, *maxGap, *feedRatio*100, encoding)

perUser := make([][]record, *users)
sentIDs := make([][]string, *users)

start := time.Now()
deadline := start.Add(*duration)

var wg sync.WaitGroup
for id := 0; id < *users; id++ {
    wg.Add(1)

    go func(id int) {
        defer wg.Done()

        rng := rand.New(rand.NewSource(*seed + int64(id)*7919))
        name := fmt.Sprintf("user-%03d", id)

        records := make([]record, 0, 1024)
        ids := make([]string, 0, 1024)

        for time.Now().Before(deadline) {
            gap := *minGap
            if *maxGap > *minGap {
                gap += time.Duration(rng.Int63n(int64(*maxGap - *minGap)))
            }
            time.Sleep(gap)

            if !time.Now().Before(deadline) {
                break
            }

            if rng.Float64() < *feedRatio {
                records = append(records, readFeed(client, *url, *feedLimit, start))
                continue
            }

            text := message(rng, *minLen, *maxLen)
            messageID := fmt.Sprintf("%s-%d-%d", name, *seed, len(ids))

            r := postMessage(client, *url, name, text, messageID, start)
            records = append(records, r)

            if !r.err && r.status == http.StatusOK {
                ids = append(ids, messageID)
            }
        }

        perUser[id] = records
        sentIDs[id] = ids
    }(id)
}

```

```

wg.Wait()
elapsed := time.Since(start)

var all []record
for _, records := range perUser {
    all = append(all, records...)
}

summary := summarise(*experiment, all, elapsed, *warmup, *users, *url)
summary["message_length_min"] = *minLen
summary["message_length_max"] = *maxLen
summary["interval_min_ms"] = minGap.Milliseconds()
summary["interval_max_ms"] = maxGap.Milliseconds()
summary["feed_ratio"] = *feedRatio

if *verify {
    var expected []string
    for _, ids := range sentIDs {
        expected = append(expected, ids...)
    }
    summary["verification"] = verifyFeed(client, *url, expected)
}

report(summary)

if *out != "" {
    writeJSON(*out, summary)
    fmt.Printf("\nwrote %s\n", *out)
}

if *series != "" {
    writeSeries(*series, all, *warmup)
    fmt.Printf("wrote %s\n", *series)
}
}

// A message of a random length, built out of whole words so it looks like text
// rather than noise.
func message(rng *rand.Rand, minLen, maxLen int) string {
    target := minLen
    if maxLen > minLen {
        target += rng.Intn(maxLen - minLen)
    }

    var b strings.Builder
    for b.Len() < target {
        if b.Len() > 0 {
            b.WriteByte(' ')
        }
        b.WriteString(words[rng.Intn(len(words))])
    }

    text := b.String()
    if len(text) > maxLen {
        text = text[:maxLen]
    }

    return text
}

func postMessage(client *http.Client, base, name, text, id string, start time.Time) record {
    payload, _ := json.Marshal(map[string]string{
        "client-name": name,
        "msg":        text,
        "id":         id,
    })

    at := time.Since(start)
    sent := time.Now()

    resp, err := client.Post(base+"/message", "application/json", bytes.NewReader(payload))
    if err != nil {
        return record{at: at, latency: time.Since(sent), err: true}
    }
    defer resp.Body.Close()

    io.Copy(io.Discard, resp.Body)

    return record{
        at:         at,
        latency:    time.Since(sent),
        status:     resp.StatusCode,
        backend:    resp.Header.Get("X-Backend"),
    }
}

```

```

    }
}

func readFeed(client *http.Client, base string, limit int, start time.Time) record {
    endpoint := base + "/feed"
    if limit > 0 {
        endpoint = fmt.Sprintf("%s?limit=%d", endpoint, limit)
    }

    at := time.Since(start)
    sent := time.Now()

    resp, err := client.Get(endpoint)
    if err != nil {
        return record{at: at, latency: time.Since(sent), feed: true, err: true}
    }
    defer resp.Body.Close()

    io.Copy(io.Discard, resp.Body)

    return record{
        at:      at,
        latency: time.Since(sent),
        status:  resp.StatusCode,
        backend: resp.Header.Get("X-Backend"),
        feed:    true,
    }
}

// -----
// Results
// -----

func summarise(name string, all []record, elapsed, warmup time.Duration, users int, url string) map[string]any {
    // The first few seconds are connection setup and JIT warm up rather than
    // steady state, so they are measured but not counted.
    var kept []record
    for _, r := range all {
        if r.at >= warmup {
            kept = append(kept, r)
        }
    }

    measured := elapsed - warmup
    if measured <= 0 {
        measured = elapsed
    }

    var posts, feeds, errors, failed int
    perBackend := map[string]int{}
    postLatencies := make([]time.Duration, 0, len(kept))
    feedLatencies := make([]time.Duration, 0, len(kept))
    allLatencies := make([]time.Duration, 0, len(kept))

    for _, r := range kept {
        switch {
        case r.err:
            errors++
        case r.status != http.StatusOK:
            failed++
        default:
            allLatencies = append(allLatencies, r.latency)
            if r.feed {
                feedLatencies = append(feedLatencies, r.latency)
            } else {
                postLatencies = append(postLatencies, r.latency)
            }
            if r.backend != "" {
                perBackend[r.backend]++
            }
        }
    }

    if r.feed {
        feeds++
    } else {
        posts++
    }
}

return map[string]any{
    "experiment": name,
    "url":        url,
    "users":      users,
}

```

```

        "duration_s":      round(elapsed.Seconds(), 2),
        "measured_s":     round(measured.Seconds(), 2),
        "requests":       len(kept),
        "messages_posted": posts,
        "feed_reads":     feeds,
        "transport_errors": errors,
        "non_200_responses": failed,
        "throughput_rps": round(float64(len(kept))/measured.Seconds(), 1),
        "latency_ms":      percentiles(allLatencies),
        "post_latency_ms": percentiles(postLatencies),
        "feed_latency_ms": percentiles(feedLatencies),
        "per_backend":     perBackend,
    }
}

func percentiles(values []time.Duration) map[string]float64 {
    if len(values) == 0 {
        return map[string]float64{}
    }

    sort.Slice(values, func(i, j int) bool { return values[i] < values[j] })

    at := func(p float64) float64 {
        index := int(p / 100 * float64(len(values)))
        if index >= len(values) {
            index = len(values) - 1
        }
        return round(float64(values[index].Microseconds())/1000, 3)
    }

    var total time.Duration
    for _, v := range values {
        total += v
    }

    return map[string]float64{
        "count": float64(len(values)),
        "mean":  round(float64(total.Microseconds())/1000/float64(len(values)), 3),
        "p50":   at(50),
        "p90":   at(90),
        "p95":   at(95),
        "p99":   at(99),
        "max":   round(float64(values[len(values)-1].Microseconds())/1000, 3),
    }
}

// Checks the run rather than timing it: every id sent must be in the feed, and
// no id may be there twice.
func verifyFeed(client *http.Client, base string, expected []string) map[string]any {
    resp, err := client.Get(base + "/feed")
    if err != nil {
        return map[string]any{"ok": false, "error": err.Error()}
    }
    defer resp.Body.Close()

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        return map[string]any{"ok": false, "error": err.Error()}
    }

    var entries []feedEntry
    if err := json.Unmarshal(body, &entries); err != nil {
        return map[string]any{"ok": false, "error": "feed is not a JSON array: " + err.Error()}
    }

    seen := make(map[string]int, len(entries))
    for _, entry := range entries {
        seen[entry.ID]++
    }

    var missing, duplicated []string
    for _, id := range expected {
        switch seen[id] {
        case 1:
        case 0:
            missing = append(missing, id)
        default:
            duplicated = append(duplicated, id)
        }
    }

    return map[string]any{
        "ok": len(missing) == 0 && len(duplicated) == 0,
    }
}

```

```

        "sent":          len(expected),
        "feed_entries": len(entries),
        "missing":      len(missing),
        "duplicated":    len(duplicated),
        "missing_examples": firstFew(missing),
        "duplicate_examples": firstFew(duplicated),
    }
}

func firstFew(items []string) []string {
    if len(items) > 5 {
        return items[:5]
    }
    return items
}

func report(summary map[string]any) {
    encoded, _ := json.MarshalIndent(summary, "", " ")
    fmt.Println(string(encoded))
}

func writeJSON(path string, value any) {
    encoded, _ := json.MarshalIndent(value, "", " ")
    if err := os.WriteFile(path, append(encoded, '\n'), 0o644); err != nil {
        fmt.Fprintf(os.Stderr, "could not write %s: %v\n", path, err)
    }
}

// One row per second: how many requests completed in that second and what the
// response time looked like inside it.
func writeSeries(path string, all []record, warmup time.Duration) {
    buckets := map[int][]time.Duration{}
    errorsPerSecond := map[int]int{}

    for _, r := range all {
        second := int(r.at.Seconds())
        if r.err || r.status != http.StatusOK {
            errorsPerSecond[second]++
            continue
        }
        buckets[second] = append(buckets[second], r.latency)
    }

    seconds := make([]int, 0, len(buckets))
    for second := range buckets {
        seconds = append(seconds, second)
    }
    for second := range errorsPerSecond {
        if _, ok := buckets[second]; !ok {
            seconds = append(seconds, second)
        }
    }
    sort.Ints(seconds)

    var b strings.Builder
    b.WriteString("second,requests,errors,mean_ms,p50_ms,p95_ms,p99_ms,warmup\n")

    for _, second := range seconds {
        values := buckets[second]
        stats := percentiles(values)

        warm := 0
        if time.Duration(second)*time.Second < warmup {
            warm = 1
        }

        fmt.Fprintf(&b, "%d,%d,%d,%g,%g,%g,%g,%d\n",
            second, len(values), errorsPerSecond[second],
            stats["mean"], stats["p50"], stats["p95"], stats["p99"], warm)
    }

    if err := os.WriteFile(path, []byte(b.String()), 0o644); err != nil {
        fmt.Fprintf(os.Stderr, "could not write %s: %v\n", path, err)
    }
}

func round(v float64, places int) float64 {
    factor := 1.0
    for i := 0; i < places; i++ {
        factor *= 10
    }
}

```

```
    return float64(int64(v*factor+0.5)) / factor  
}
```

`sources/loadgen/main.go`

Akshat Gupta, 12340160. Every number in this report was measured on stu68_sys1–4 by the load generator in section 7, and every figure is drawn from the files those runs produced.